



Wrocław University
of Science and Technology

OPTIMIZATION METHODS
PROJECT GROUP 1

Task 0: Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods

**The Permutation Flowshop Scheduling Problem
— Project's Report —**

Student

Maxime LUCE
Student ID: 294620

Teacher

Marcin KOMARNICKI

Academic year 2025/2026 – March 30, 2026

Contents

1	The Permutation Flowshop Scheduling Problem	3
1.1	General overview	3
1.2	Specifics	4
1.3	Objectives	4
1.4	Constraints	5
1.5	Chosen problem representation	5
1.6	Solved example	6
2	Theoretical review	8
2.1	Solving an Optimization problem : Exploration and Exploitation	8
2.2	Random Search	8
2.3	Greedy Algorithm	9
2.4	Evolutionary Algorithms	11
2.4.1	Overview	11
2.4.2	Selection	11
2.4.3	Mutation	12
2.4.4	Crossover	13
2.4.5	Elitism	18
2.5	Simulated Annealing	18
3	Implementation and protocol	19
3.1	Description of the protocol	19
3.1.1	Hardware description and environment	19
3.1.2	Stop condition	20
3.1.3	Measuring computation time	20
3.2	Main structure	20
3.3	Unit tests	22
4	Finding the optimal parameters	22
4.1	Description of the different cases	22
4.1.1	Evolutionary Algorithm	23
4.1.2	Simulated Annealing	25
4.2	Running experiment	26
4.2.1	Evolutionary Algorithm	26
4.2.2	Simulated Annealing	37
4.3	Summary of the parameters	40



5 Solving the permutation flowshop scheduling problem	40
5.1 Comparison of all methods on every instances	40
5.2 Convergence of Evolutionary Algorithm	41
Conclusion	43
References	43
Appendixes	45
List of Tables	45
List of Figures	47
List of Algorithms	47
List of Listings	47

Introduction

This document aims to present the main results of the project on *Permutation Flowshop Scheduling Problem* (PFSP), which is an optimization problem. This project is an introduction to evolutionary algorithms and their comparison to non-evolutionary methods.

Some optimization problems are so hard that we can't find any algorithm, *i.e.* a finite sequence of instructions in order to get the best solution, to solve it. In complexity theory, we can define an NP problem by two properties: we can easily verify a solution – polynomial complexity – but we cannot design a process to build an efficient solution. The PFSP is an example of NP-problem, so we need to explore different approaches to find the best solution possible. The main alternative consists in using *evolutionary algorithms*, and especially *genetic algorithms* that reproduce the natural genetical process, in order to generate better and better solutions across generations by using selection and mutation rules. Another alternative is the model through physical process such as the annealing of a metal, which leads to the *simulated annealing*. Our goal is to study all these methods and their implementation for the PFSP in order to compare them and find the best way to solve the PFSP.

We will start by presenting the PFSP and giving a solved example in order to get familiar with the problem 1. Then, we will provide some theoretical review on formalizing an optimization problem and on methods to solve it. We will also present our implementation and protocol for our numerical analysis 3. Finally, we will present our method to determine the best set of parameters for our algorithms 4 and, to finish, we will present our solutions to the PFSP 5.

1 The Permutation Flowshop Scheduling Problem

1.1 General overview

The PFSP is defined by a set of M machines, J jobs and a matrix $P \in \mathcal{M}_{M,J}(\mathbb{R})$, where $p_{m,j} = p(m,j)$ represents the processing times of job j on machine m with $j \in \llbracket 1; J \rrbracket$ and $m \in \llbracket 1; M \rrbracket$.

The constraints are that each job must be processed on each machine. Moreover, the order of machines on which jobs are processed is predefined, which means that we cannot start processing a job on a machine until the job is done on the previous machine, and the order is the same for all jobs. Finally, each machine can process only one job at a time.

We aim to find a job sequence $x = \langle x_1, \dots, x_n \rangle$ to be processed on each machine which minimizes the total flow time. We define the total flow time as the sum of the completion times of each job on the last machine. Hence, the objective function f that compute the total flow is defined as:

$$f(x) = \sum_{j=1}^J c(x_j, M), \quad (1)$$

where $c(x_j, m)$ is the completion time of the x_j -th job on the m -th machine. This quantity can be calculated with



$$c(x_j, m) = \begin{cases} p(x_j, m) & , \text{ if } j = 1 \text{ and } m = 1 \\ c(x_j, m - 1) + p(x_j, m) & , \text{ if } j = 1 \text{ and } m > 1 \\ c(x_{j-1}, m) + p(x_j, m) & , \text{ if } j > 1 \text{ and } m = 1 \\ \max \{c(x_{j-1}, m), c(x_j, m - 1)\} + p(x_j, m) & , \text{ if } j > 1 \text{ and } m > 1 \end{cases} \quad (2)$$

where $p(x_j, m)$ is the processing time on job x_j on machine m .

1.2 Specifics

Regarding (2) and the presence of a maximum, the PFSP is a non-linear problem that is NP-hard provided that $M > 3$ (see [1]). Indeed, if we consider a sequence of J jobs, there are $J!$ possible sequences.

For our topic, we will consider that all computation times are constant and known, there is no stochastic process involved.

Furthermore, we will assume that a job cannot be interrupted and that machines are always available to run a job. We will also assume that we need no time to start a job on a machine, it is instantaneous.

1.3 Objectives

Our main goal is determine the best sequence x^* that minimizes the objective function (1), i.e. minimize the total flow time or the total completion time.

In order to achieve that, our goal is to describe, implement and compare several methods (Table 1) : the random search, the greedy algorithm, evolutionary algorithms and simulated annealing.

Finally, this comparison and the analysis will let us determine the best set of parameters for each method and in which situation each method is better.

Method	Complexity	Description
Random Search	$\mathcal{O}(E \cdot J \cdot M)$	The random search iteratively generates completely random permutations of jobs and evaluates them. It retains the best solution found but lacks any guided search or memory mechanism.
Greedy Algorithm	$\mathcal{O}(J^2 \cdot M)$	The greedy algorithm is a constructive heuristic that builds a single schedule step-by-step by making the locally optimal choice at each step. While extremely fast, it is highly likely to get trapped in poor local optimum.
Evolutionary Algorithm (EA)	$\mathcal{O}(G \cdot P \cdot J \cdot M)$	The EA is a population-based metaheuristic inspired by biological evolution : parent selection, mutation and crossover and generations phenomenon.
Simulated Annealing (SA)	$\mathcal{O}(E \cdot J \cdot M)$	The simulated annealing is a metaheuristic inspired by the physical annealing of metals. It uses a cooling rate to probabilistically accept worse solutions during first iterations (exploration), and then going to the local optimum (exploitation) as the temperature drops.

Table 1: Description of methods to be implemented. Each method is presented by its name, computational complexity (where J is the number of jobs, M the number of machines, E the total evaluations, P the population size, and G the number of generations), and a short description.

1.4 Constraints

As mentioned in 1.1, each job visits all the machines in the same order (coming from the given sequence) and we assume that the machine order is fixed.

We also assume that we can only proceed one job at a time on a specific machine and one machine for a specific job at a time.

As we will implement the methods in Python, we have a strong constraint on optimization: our code must be able to run all the given instances of different size of problem to run our experiments. Furthermore, our code must be modular and flexible in order to use it for further development and projects.

1.5 Chosen problem representation

We will follow the mathematical definition of the PFSP of a sequence x as a sequence of numbers in a permutation of J jobs. Therefore, we can write x as $\langle x_i, \dots, x_n \rangle$. This implementation allows us to easily use it in our methods, especially with EA and all operators such as swap, inversion or crossover.

For example, we can encode a sequence x like $\langle 1, 3, 4, 2 \rangle$. It means that the first job processed on each machine is the first one, the second is the third, and so on.



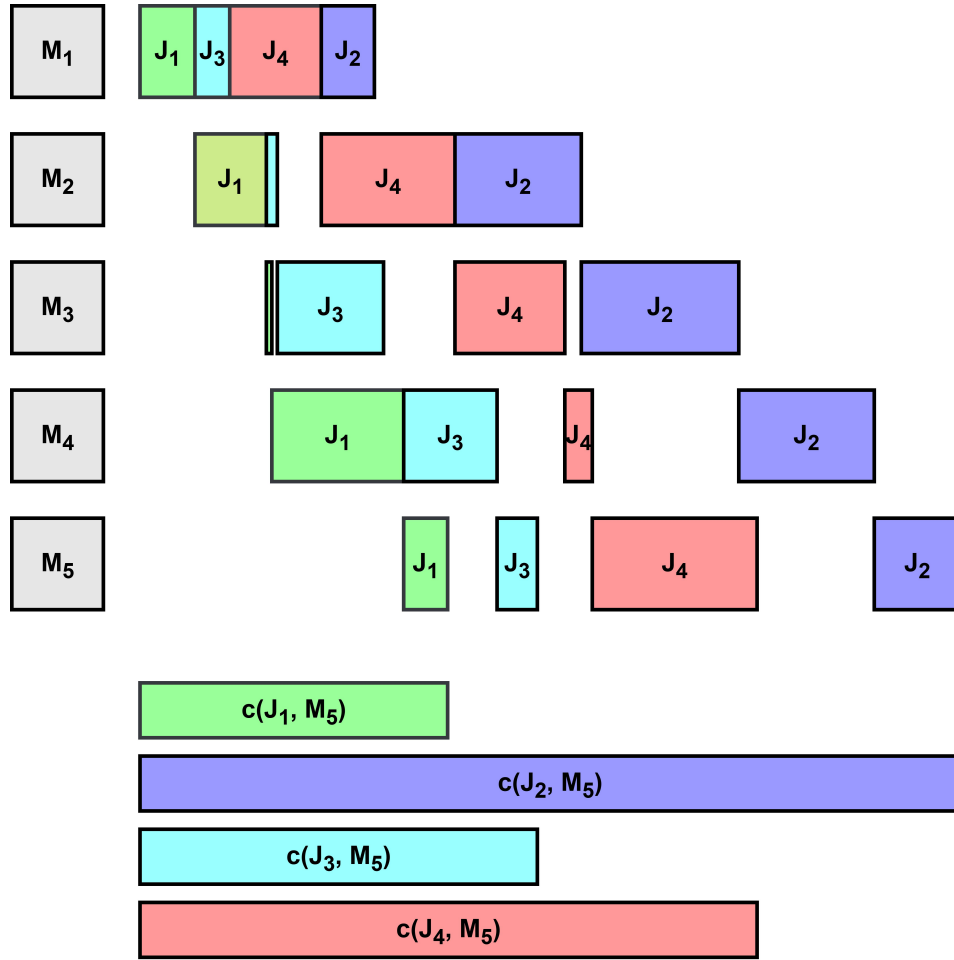


Figure 1: A representation of an example sequence. (Source: [2])

1.6 Solved example

Let's consider the following problem with 4 jobs and 5 machines and the given computation times matrix (3).

$$P = \begin{pmatrix} 30 & 29 & 19 & 50 \\ 39 & 69 & 6 & 73 \\ 3 & 86 & 58 & 60 \\ 72 & 74 & 51 & 15 \\ 24 & 47 & 22 & 90 \end{pmatrix} \quad (3)$$

For this problem, we know the optimal solution, which is the sequence $x^* = \langle 1, 3, 4, 2 \rangle$.

By using equation (1), we can compute the total flow time:

$$f(x^*) = c(1, 5) + c(3, 5) + c(4, 5) + c(2, 5).$$

Now, we use (2) to evaluate each cost. We present the calculations in the reverse order necessary to understand the dependencies between the relationships.



- **Calculation of $c(1, 5)$**

- We can in fact prove by recurrence that, for all $n \in \mathbb{N}$, we have $c(1, n) = \sum_{j=1}^n p(1, j)$.
- For $n = 5$, we have indeed:

$$\begin{aligned}
 c(1, 5) &= c(1, 4) + p(1, 5) \\
 &= c(1, 3) + p(1, 4) + p(1, 5) \\
 &= \dots \\
 &= p(1, 1) + p(1, 2) + p(1, 3) + p(1, 4) + p(1, 5) \\
 &= 30 + 39 + 3 + 72 + 24
 \end{aligned}$$

$$\boxed{c(1, 5) = 168}$$

- **Calculation of $c(3, 5)$**

- $c(1, 2) = p(1, 1) + p(1, 2) = 40 + 39 = 69$
- $c(3, 1) = c(1, 1) + p(3, 1) = p(1, 1) + p(3, 1) = 30 + 19 = 49$
- $c(3, 2) = \max \{c(1, 2), c(3, 1)\} + p(3, 2) = c(1, 2) + p(3, 2) = 69 + 6 = 75$
- $c(3, 3) = \max \{c(1, 3), c(3, 2)\} + p(3, 3) = c(3, 2) + p(3, 3) = 75 + 58 = 133$
- $c(3, 4) = \max \{c(1, 4), c(3, 3)\} + p(3, 4) = c(1, 4) + p(3, 4) = 144 + 51 = 195$
- $c(3, 5) = \max \{c(1, 5), c(3, 4)\} + p(3, 5) = c(3, 4) + p(3, 5) = 195 + 22 = 217$
- $\boxed{c(3, 5) = 217}$

- **Calculation of $c(4, 5)$**

- $c(4, 1) = c(3, 1) + p(4, 1) = 49 + 50 = 99$
- $c(4, 2) = \max \{c(3, 2), c(4, 1)\} + p(4, 2) = c(4, 1) + p(4, 2) = 99 + 73 = 172$
- $c(4, 3) = \max \{c(3, 3), c(4, 2)\} + p(4, 3) = c(4, 2) + p(4, 3) = 172 + 60 = 232$
- $c(4, 4) = \max \{c(3, 4), c(4, 3)\} + p(4, 4) = c(4, 3) + p(4, 4) = 232 + 15 = 247$
- $\boxed{c(4, 5) = 247}$

- **Calculation of $c(2, 5)$**

- $c(2, 1) = c(4, 1) + p(2, 1) = 99 + 29 = 128$
- $c(2, 2) = \max \{c(4, 2), c(2, 1)\} + p(2, 2) = c(4, 2) + p(2, 2) = 172 + 69 = 241$
- $c(2, 3) = \max \{c(4, 3), c(2, 2)\} + p(2, 3) = c(2, 2) + p(2, 3) = 241 + 86 = 327$
- $c(2, 4) = \max \{c(4, 4), c(2, 3)\} + p(2, 4) = c(2, 3) + p(2, 4) = 327 + 74 = 401$
- $c(2, 5) = \max \{c(4, 5), c(2, 4)\} + p(2, 5) = c(2, 4) + p(2, 5) = 401 + 47 = 448$
- $\boxed{c(2, 5) = 448}$

- Finally, we get $\boxed{f(x^*) = 168 + 217 + 337 + 448 = 1170}$



2 Theoretical review

2.1 Solving an Optimization problem : Exploration and Exploitation

When it comes to find an optimal solution to a given problem, we can first try to describe well our problem. They are two main categories of problem : *combinatorial* and *topological*. In optimization, a *topological* problem seeks the best continuous distribution of material within a physical space (such as designing the most efficient shape for a mechanical component), whereas a *combinatorial* problem involves finding the optimal discrete arrangement from a finite set of possibilities (such as finding the best sequence of tasks in the PFSP).

A general n -dimensional optimization problem is described by an objective function f

$$f : \mathcal{D}_f \subset \mathbb{R}^n \rightarrow R, \quad (4)$$

where $\mathcal{D}_f$ is the solution search space. Our objective is therefore to find x^* which can be the maximum or the minimum of f , let's assume we are looking for a maximum (even though it means changing f to $-f$).

$$x^* = \arg \max_{x \in \mathcal{D}_x \subset \mathcal{D}_f} f(x), \quad (5)$$

where $\mathcal{D}_x$ is the feasible set.

In the permutation flowshop scheduling problem, the objective function is defined (1) and we can observe that finding the maximum of this function is really hard because this function is not linear, nor convex. Therefore, as it is a NP-hard problem, we cannot use algorithms to solve this problem. Instead, we will use several methods to look for the best solution possible in the feasible set. We will now present these approaches: the random search 2.2, the greedy algorithm 2.3, the evolutionary algorithms 2.4 and the simulated annealing approach 2.5.

2.2 Random Search

The random search is the most intuitive and easiest approach to find a good solution. We will randomly generate a solution, in the feasible set, we will evaluate its score and we will save the best score and the best solution found during the number of evaluations.

The [Algorithm 1](#) presents the pseudo-code for this approach.

Algorithm 1 Random Search for the Permutation Flowshop Scheduling Problem (PFSP)**Require:** *num_jobs* (number of jobs), *max_evaluations* (maximum number of evaluations)**Ensure:** *best_individual* (the best sequence found)

```

1: best_individual  $\leftarrow$  Null
2: for  $i = 1$  to max_evaluations do
3:   // Generate a random sequence
4:   seq  $\leftarrow$  Generate a random permutation of num_jobs jobs
5:   ind  $\leftarrow$  New individual with sequence seq
6:   // Evaluation
7:   Evaluate ind (calculate fitness/makespan for the PFSP)
8:   // Update the best solution
9:   if best_individual is Null or ind.fitness < best_individual.fitness then
10:    best_individual  $\leftarrow$  ind
11:   end if
12: end for
13: return best_individual

```

2.3 Greedy Algorithm

The greedy algorithm is an approach where the problem is solved step by step by selecting the best possible choice at each moment. We name this approach "greedy" because it needs to evaluate each possible choice at each step. This number of evaluations is decreasing but can still be very high at the beginning of the algorithm.

Unlike random search and others methods, this algorithm is not random: each run will give us the same solution. Therefore, there is no need to run the algorithm multiple times to average the results.

The [Algorithm 2](#) presents the pseudo-code of this algorithm, applied to the PFSP problem.

Algorithm 2 Constructive Greedy Heuristic for the PFSP**Require:** N (number of jobs), M (number of machines), p (processing times matrix)**Ensure:** *greedy_individual* (the sequence constructed greedily)

```

1: function GETCOMPLETIONTIME(partial_sequence)
2:   if partial_sequence is empty then
3:     return 0
4:   end if
5:    $J \leftarrow$  length of partial_sequence
6:   Initialize completion time matrix  $C$  of size  $M \times J$  with zeros
7:    $C_{0,0} \leftarrow p_{0,partial\_sequence[0]}$ 
8:   for  $j = 1$  to  $J - 1$  do
9:      $C_{0,j} \leftarrow C_{0,j-1} + p_{0,partial\_sequence[j]}$ 
10:  end for
11:  for  $m = 1$  to  $M - 1$  do
12:     $C_{m,0} \leftarrow C_{m-1,0} + p_{m,partial\_sequence[0]}$ 
13:  end for
14:  for  $m = 1$  to  $M - 1$  do
15:    for  $j = 1$  to  $J - 1$  do
16:       $C_{m,j} \leftarrow \max(C_{m,j-1}, C_{m-1,j}) + p_{m,partial\_sequence[j]}$ 
17:    end for
18:  end for
19:  return  $C_{M-1,J-1}$ 
20: end function

// Main execution of the greedy algorithm
21: unscheduled_jobs  $\leftarrow \{0, 1, \dots, N - 1\}$ 
22: final_sequence  $\leftarrow []$  // Empty list
23: while unscheduled_jobs is not empty do
24:   best_job  $\leftarrow$  Null
25:   best_completion_time  $\leftarrow \infty$ 
26:   for each job in unscheduled_jobs do
27:     test_sequence  $\leftarrow$  final_sequence  $\cup [job]$ 
28:     completion_time  $\leftarrow$  GETCOMPLETIONTIME(test_sequence)
29:     if completion_time < best_completion_time then
30:       best_completion_time  $\leftarrow$  completion_time
31:       best_job  $\leftarrow$  job
32:     end if
33:   end for
34:   Append best_job to final_sequence
35:   Remove best_job from unscheduled_jobs
36: end while
37: greedy_individual  $\leftarrow$  New Individual with sequence final_sequence
38: Evaluate greedy_individual (calculate full fitness)
39: return greedy_individual

```

2.4 Evolutionary Algorithms

2.4.1 Overview

Based on biological evolution, the evolutionary algorithms reproduce this approach to solve such optimization problem. This approach has several steps such as: reproduction, mutation, recombination, selection and elitism (Figure 2). These steps are indeed the main parameters of the method: what is the size of the population? How many generations should we consider? What kind of mutation are we using? With what probability? How do we model the recombination or the crossover? How do we select individual (the member of the population) and do we apply elitism?

All these parameters have an influence on the ratio between exploration and exploitation as discussed in 2.1 and will now succinctly describe these parameters.

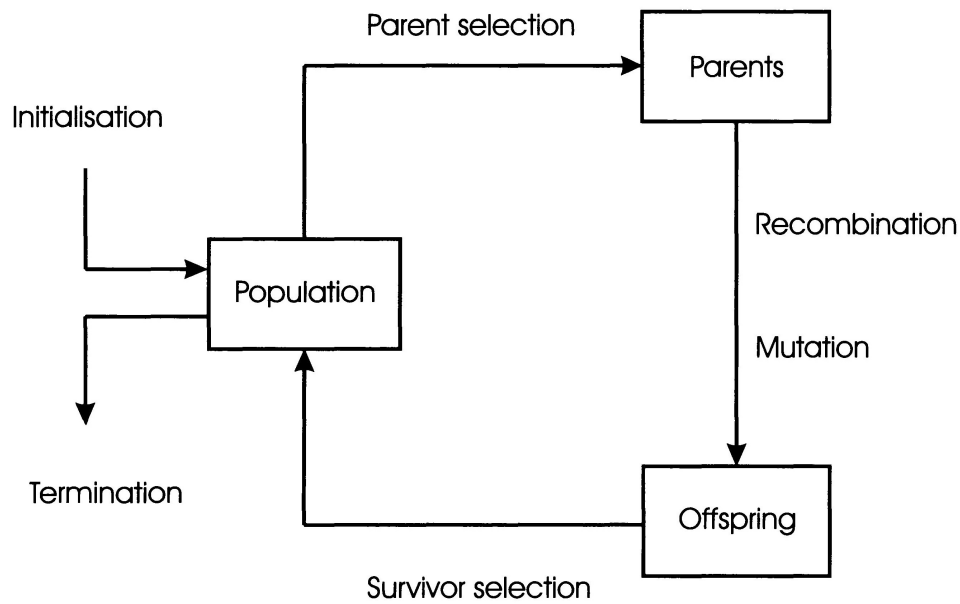


Figure 2: The general scheme of an evolutionary algorithm as a flow-chart. All the steps of the EA are represented: we initialize a population (randomly), we select the parents, it might be mutation and recombination to form the child to create the new generation of the population. We loop until we run enough generations to have a good solution. The elitism is not represented in this scheme. (Source: [3])

2.4.2 Selection

They are two main selection methods: the *roulette* and the *tournament* that we will now present and analyze.

Roulette Even if this approach is fair, it is highly dependent on the values of the optimized function (because if we add a huge number to the objective function, we will get almost equal probability so an equiprobable situation but the maximum is still the same) so it makes controlling the convergence hard. Finally, this selection's method imposes that fitness must be positive.

For all these reasons, and as state-of-the-art GAs do not use roulette wheel [4], we will not implement this method but the *tournament* instead.

Tournament The *tournament* method combines both randomness and selection in order to avoid the issue of the roulette. We start indeed by choosing k individuals randomly and then we select among these k individuals the best individual, who is named "winner of the tournament".

On the one hand, this method helps to control promote best individuals with high values of k , which increase the convergence. On the other hand, it helps to promote diversity with low k -value, which give a chance for lower-fitting individuals [4].

Therefore, we will implement this selection method and the number k will be one of our parameters.

2.4.3 Mutation

As we use permutation to model the problem, we consider two mutation type: the *swap* and the *inversion*.

Swap This operation consists of swapping two random elements of the permutation, like two genes in a chromosome. For example, if we take the permutation $\langle 2, 4, 6, 9, 1, 3, 8 \rangle$, we can choose randomly two elements within the permutation. Let's assume we choose 2 and 3, then the mutated sequence become $\langle 3, 4, 6, 9, 1, 2, 8 \rangle$.

The **Algorithm 3** present the pseudo-code of this mutation method.

Algorithm 3 Swap Mutation Operator for the PFSP

Require: *individual* (a given sequence of jobs), p_m (mutation probability)

Ensure: Mutated *individual*

- 1: $r \leftarrow$ Generate a random number in $[0, 1)$
 - 2: **if** $r < p_m$ **then**
 - 3: $N \leftarrow$ length of *individual.sequence*
 - 4: Randomly select two distinct indices idx_1 and idx_2 from $\{0, 1, \dots, N - 1\}$
 - 5: // Swap the tasks at the selected indices
 - 6: $temp \leftarrow individual.sequence[idx_1]$
 - 7: $individual.sequence[idx_1] \leftarrow individual.sequence[idx_2]$
 - 8: $individual.sequence[idx_2] \leftarrow temp$
 - 9: **end if**
 - 10: **return** *individual*
-

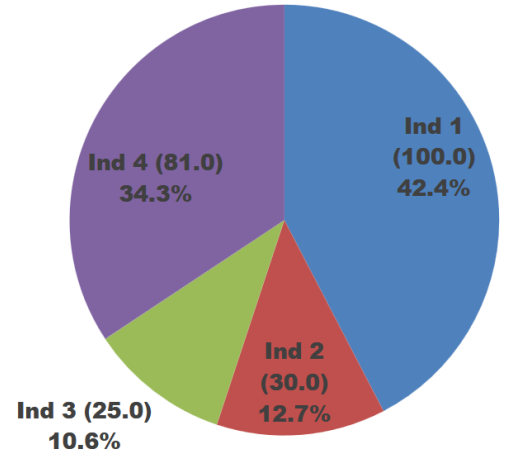


Figure 3: Roulette selection scheme.
(Source: [4])

Inversion We will implement another mutation method, the *inversion*. In these mutations, we also select two random positions in the sequence but instead of swapping, we reverse all elements in between these two positions. Let's take again the example of the sequence $\langle 2, 4, 6, 9, 1, 3, 8 \rangle$ and let's also assume that we choose randomly 2 and 3. Then, the mutated sequence become $\langle 3, 1, 9, 6, 4, 2, 8 \rangle$. The **Algorithm 4** presents the pseudo-code of this mutation based on the swap-one.

Algorithm 4 Inversion Mutation Operator for the PFSP

Require: *individual* (a given sequence of jobs), p_m (mutation probability)

Ensure: Mutated *individual*

```

1:  $r \leftarrow$  Generate a random number in  $[0, 1)$ 
2: if  $r < p_m$  then
3:    $N \leftarrow$  length of individual.sequence
4:   Randomly select two distinct indices  $idx_1$  and  $idx_2$  from  $\{0, 1, \dots, N - 1\}$ 
5:   if  $idx_1 > idx_2$  then
6:     Swap  $idx_1$  and  $idx_2$  // Ensure  $idx_1 < idx_2$ 
7:   end if
8:   // Reverse the subsequence between  $idx_1$  and  $idx_2$  inclusive
9:   while  $idx_1 < idx_2$  do
10:     $temp \leftarrow individual.sequence[idx_1]$ 
11:     $individual.sequence[idx_1] \leftarrow individual.sequence[idx_2]$ 
12:     $individual.sequence[idx_2] \leftarrow temp$ 
13:     $idx_1 \leftarrow idx_1 + 1$ 
14:     $idx_2 \leftarrow idx_2 - 1$ 
15:   end while
16: end if
17: return individual

```

2.4.4 Crossover

The *crossover* (or the *recombination* on **Figure 2**) aims to create the descendant, or the offspring, or the children of the two parents. We will present and compare three crossover : the *Ordered Crossover* (OX), the *Partial Matched Crossover* (PMX) and the *Cycle Crossover* (CX).

Ordered Crossover In order to run the *ordered crossover*, we start by randomly pick two positions on one parent's sequence. Then, we copy the subsequence between these two positions to the offspring. All others positions are then copy from the second parent (in the order of the parent's sequence after removing the previous subsequence's position).

Let's take the example of parent P_1 with the sequence $\langle 2, 5, 3, 4, 8, 9, 1, 7, 6 \rangle$ and parent P_2 with the sequence $\langle 1, 9, 6, 2, 3, 4, 5, 7, 8 \rangle$. Let's assume that we pick 5 and 1 number from parent P_1 . Then the ordered crossover operation, noted $\mathcal{R}_{OX}$, can be represented as follows:

$$\langle 2, \underbrace{5, 3, 4, 8, 9, 1}_{\text{we copy this subsequence}}, 7, 6 \rangle \mathcal{R}_{OX} \langle 1, 9, 6, 2, \underbrace{3, 4, 5, 7, 8}_{\text{copied subsequence}} \rangle = \langle 6, \underbrace{5, 3, 4, 8, 9, 1}_{\text{copied subsequence}}, 2, 7 \rangle \quad (6)$$

The **Algorithm 5** present the pseudo-code of this crossover which is based on the function "GenerateChild". This function take care of the copy and the extraction of the task (or gene) from the second



parent.

Algorithm 5 Order Crossover (OX) Operator for the PFSP

Require: $parent_1, parent_2$ (two parent individuals)

Ensure: $child_1, child_2$ (two offspring individuals)

```

1: function GENERATECHILD( $p_1, p_2$ )
2:    $N \leftarrow \text{length of } p_1.\text{sequence}$ 
3:   Randomly select two distinct indices  $start$  and  $end$  from  $\{0, 1, \dots, N - 1\}$ 
4:   if  $start > end$  then
5:     Swap  $start$  and  $end$  // Ensure  $start < end$ 
6:   end if
7:   Initialize array  $child\_seq$  of size  $N$  with Null
8:   // Copy the subsequence from the first parent
9:   for  $i = start$  to  $end$  do
10:     $child\_seq[i] \leftarrow p_1.\text{sequence}[i]$ 
11:  end for
12:  // Extract genes from the second parent not yet in the child
13:   $p_2\_filtered \leftarrow []$  // Empty list
14:  for each  $gene$  in  $p_2.\text{sequence}$  do
15:    if  $gene \notin child\_seq$  then
16:      Append  $gene$  to  $p_2\_filtered$ 
17:    end if
18:  end for
19:  // Fill the remaining Null positions from left to right
20:   $idx \leftarrow 0$ 
21:  for  $i = 0$  to  $N - 1$  do
22:    if  $child\_seq[i]$  is Null then
23:       $child\_seq[i] \leftarrow p_2\_filtered[idx]$ 
24:       $idx \leftarrow idx + 1$ 
25:    end if
26:  end for
27:  return New Individual with sequence  $child\_seq$ 
28: end function

  // Generate two children by swapping parent roles
29:  $child_1 \leftarrow \text{GENERATECHILD}(parent_1, parent_2)$ 
30:  $child_2 \leftarrow \text{GENERATECHILD}(parent_2, parent_1)$ 
31: return  $child_1, child_2$ 

```

Partial matched crossover The *partial matched crossover* is an extension of the ordered crossover. We also starts by choosing randomly two elements for one parent in order to get a subsequence. Then, we map the numbers of the corresponding position in the other parent. Finally, the mapping subsequence is copied to the offspring sequence and completed with the remaining mapped sequence of the other parent.

Let's take the example of parent P_1 with the sequence $\langle 7, 5, 3, 1, 9, 8, 6, 4, 2 \rangle$ and parent P_2 with the sequence $\langle 1, 3, 5, 2, 4, 6, 8, 9, 7 \rangle$. Let's assume that we pick the subsequence between positions 4 and 6 (so between 1 and 8) from parent P_1 . This selection establishes the mapping $1 \longleftrightarrow 2$, $9 \longleftrightarrow 4$, and $8 \longleftrightarrow 6$ with the corresponding elements in P_2 as represented as follows:



$$\begin{array}{ccc}
 & \langle 7, 5, 3, \color{red}{1}, \color{red}{9}, \color{red}{8}, 6, 4, 2 \rangle & \\
 \text{Mapping operation} \longrightarrow & \begin{array}{c} \uparrow \uparrow \uparrow \\ \downarrow \downarrow \downarrow \end{array} & \\
 & \langle 1, 3, 5, \color{red}{2}, \color{red}{4}, \color{red}{6}, 8, 9, 7 \rangle &
 \end{array}$$

Then the partially mapped crossover operation, noted $\mathcal{R}_{PMX}$, can be represented as follows:

$$\langle 7, 5, 3, \underbrace{\color{red}{1}, \color{red}{9}, \color{red}{8}}_{\text{we copy this subsequence}}, 6, 4, 2 \rangle \mathcal{R}_{PMX} \overset{2}{\langle \cancel{1}, 3, 5, 2, 4, 6, \overset{6}{\cancel{8}}, \overset{4}{\cancel{9}}, 7 \rangle} = \langle 2, 3, 5, \underbrace{\color{red}{1}, \color{red}{9}, \color{red}{8}}_{\text{copied subsequence}}, 6, 4, 7 \rangle \quad (7)$$

Of course, we can do the same operation considering the parent P_2 in order to get two children. The main process is described in the pseudo-code [Algorithm 6](#).

Algorithm 6 Partially Mapped Crossover (PMX) Operator for the PFSP**Require:** $parent_1, parent_2$ (two parent individuals)**Ensure:** $child_1, child_2$ (two offspring individuals)

```

1: function GENERATECHILD( $p_1, p_2$ )
2:    $N \leftarrow \text{length of } p_1.\text{sequence}$ 
3:   Randomly select two distinct indices  $start$  and  $end$  from  $\{0, 1, \dots, N - 1\}$ 
4:   if  $start > end$  then
5:     Swap  $start$  and  $end$  // Ensure  $start < end$ 
6:   end if
7:   Initialize array  $child\_seq$  of size  $N$  with Null
8:   Initialize an empty dictionary  $mapping$ 
9:   // 1. Copy the mapping section from  $p_2$ 
10:  for  $i = start$  to  $end$  do
11:     $child\_seq[i] \leftarrow p_2.\text{sequence}[i]$ 
12:  end for
13:  // 2. Create the mapping dictionary:  $p_2$  value  $\rightarrow p_1$  value
14:  for  $i = start$  to  $end$  do
15:     $mapping[p_2.\text{sequence}[i]] \leftarrow p_1.\text{sequence}[i]$ 
16:  end for
17:  // 3. Fill the rest with  $p_1$ , resolving conflicts using the mapping
18:  for  $i = 0$  to  $N - 1$  do
19:    if  $i < start$  or  $i > end$  then // Ignore the already filled central section
20:       $val \leftarrow p_1.\text{sequence}[i]$ 
21:      // Trace the mapping chain until a valid, non-duplicate job is found
22:      while  $val$  exists as a key in  $mapping$  do
23:         $val \leftarrow mapping[val]$ 
24:      end while
25:       $child\_seq[i] \leftarrow val$ 
26:    end if
27:  end for
28:  return New Individual with sequence  $child\_seq$ 
29: end function

// Generate two children by swapping parent roles
30:  $child_1 \leftarrow \text{GENERATECHILD}(parent_1, parent_2)$ 
31:  $child_2 \leftarrow \text{GENERATECHILD}(parent_2, parent_1)$ 
32: return  $child_1, child_2$ 

```

Cycle Crossover The *cycle crossover* aims to guarantee that each task of the offspring comes from the same exact position than one of the two parents. To do so, we start by taking the first task of the parent 1 and we copy it to the offspring. Then, we look at the same position of the second parent and we search the position of that task in the parent 1. We then continue the same process until we get back to the starting point (when we finish the cycle).

Let's take the example given in [2] where we have a first parent $P_1 = \langle 2, 7, 4, 3, 1, 5, 6, 9, 8 \rangle$ and a second parent $P_2 = \langle 1, 2, 3, 4, 5, 6, 7, 8, 9 \rangle$. The first association of the cycle is therefore $1 \longleftrightarrow 2$. But 1 in parent 1 is in relation with 5, and then 6 and then finally 7. By doing the same with other positions,

we are able to get the full cycle (1 2 7 6 5 1) and create the offspring (Figure 4). The pseudo-code of this feature is presented in Algorithm 7.

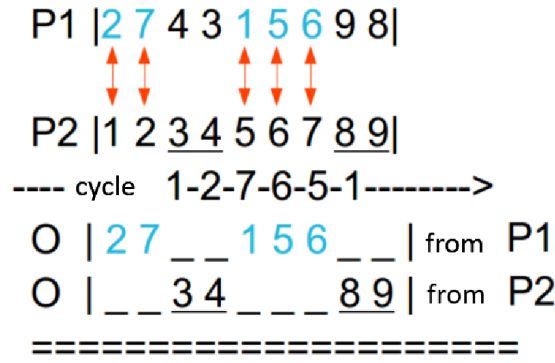


Figure 4: Example of an CX crossover. (Source: [2])

Algorithm 7 Cycle Crossover (CX) Operator for the PFSP

Require: $parent_1, parent_2$ (two parent individuals)

Ensure: $child_1, child_2$ (two offspring individuals)

```

1: function GENERATECHILD( $p_1, p_2$ )
2:    $N \leftarrow$  length of  $p_1.sequence$ 
3:   Initialize array  $child\_seq$  of size  $N$  with Null
4:    $start\_idx \leftarrow 0$ 
5:    $curr\_idx \leftarrow start\_idx$ 
6:   // Trace the cycle starting from the initial index
7:   repeat
8:     // Copy the job from the first parent
9:      $child\_seq[curr\_idx] \leftarrow p_1.sequence[curr\_idx]$ 
10:    // Find the corresponding job in the second parent
11:     $val\_in\_p2 \leftarrow p_2.sequence[curr\_idx]$ 
12:    // Find the index of this job in the first parent
13:     $curr\_idx \leftarrow$  index of  $val\_in\_p2$  in  $p_1.sequence$ 
14:  until  $curr\_idx = start\_idx$  // Stop when the cycle is complete
15:  // Fill the remaining gaps with jobs from  $p_2$ 
16:  for  $i = 0$  to  $N - 1$  do
17:    if  $child\_seq[i]$  is Null then
18:       $child\_seq[i] \leftarrow p_2.sequence[i]$ 
19:    end if
20:  end for
21:  return New Individual with sequence  $child\_seq$ 
22: end function

// Generate two children by swapping parent roles
23:  $child_1 \leftarrow$  GENERATECHILD( $parent_1, parent_2$ )
24:  $child_2 \leftarrow$  GENERATECHILD( $parent_2, parent_1$ )
25: return  $child_1, child_2$ 

```

2.4.5 Elitism

During the selection process, we can apply *elitism* by copying the best individuals without any mutation. Therefore, we can have a parameter called "elitism" that represents the number of these copy. To do so, we first order the population and take the "elitism" size best ones and then put them directly in the next generation.

2.5 Simulated Annealing

The *simulated annealing* is based on the annealing's process in metallurgy. In this process, we aim to create a more solid material by alternating slow cooling and heating (annealing) in order to relax the micro-fissure within the material and, thus, going in a more stable form of the material, and therefore minimize the total energy of the material.

Therefore, by analogy, the algorithm's process can be described by : first, we choose a random initial state in the feasible set. This state is characterized by its total energy E_0 (which is in fact evaluated by the objective function) and we use the fictive parameter T as for the temperature in the real annealing. At each step, we change slightly the state (in the *neighborhood*) and we get therefore a difference of energy ΔE . If $\Delta E < 0$, then the system has less energy so is more stable so we get that change. Otherwise, if $\Delta E > 0$, then the neighbor is worse but we still accept it with a probability $\exp\left(-\frac{\Delta E}{T}\right)$. Then, we continue to iterate until we reach the maximal number of iterations or if we obtain a pretty good solution.

We now need to describe how the temperature will change, we can set it piecewise constant or following a law of evolution. The most common is the geometric one [5], we can therefore write:

$$T_{i+1} = \lambda T_i, \quad (8)$$

where $\lambda < 1$ (otherwise it is constant) and we usually take $\lambda = 0,99$ [6].

This method has the advantage that, under some hypothesis, we can prove that this process will converge in probability [7]. But the huge disadvantage of this approach is that it is very hard to determine the good parameters to fit the model.

In fact, the temperature has a huge impact: with high temperature, the system can easily move in the feasible set because $\exp\left(-\frac{\Delta E}{T}\right)$ is close to 1. However, with low temperature, the changes that decrease the energy of the system are accepted but others are also accepted which allow us not to be blocked inside a local minimum.

Algorithm 8 Simulated Annealing for the PFSP**Require:** s_0 (initial solution), $k_{\max}$ (maximum iterations), $e_{\max}$ (target minimum energy/makespan)**Ensure:** g (best solution found)

```

1:  $s \leftarrow s_0$  // Current solution
2:  $g \leftarrow s_0$  // Global best solution
3:  $e \leftarrow E(s)$  // Energy (fitness) of current solution
4:  $m \leftarrow E(g)$  // Energy of global best
5:  $k \leftarrow 0$ 
6: while  $k < k_{\max}$  and  $e > e_{\max}$  do
7:    $s_n \leftarrow$  Randomly select a neighbor of  $s$  uniformly
8:    $e' \leftarrow E(s_n)$ 
9:   // Acceptance criterion
10:  if  $e' < e$  or  $\text{Random}(0, 1) < P(e' - e, \text{temp}(k/k_{\max}))$  then
11:     $s \leftarrow s_n$ 
12:     $e \leftarrow e'$ 
13:  end if
14:  // Update global best
15:  if  $e < m$  then
16:     $g \leftarrow s$ 
17:     $m \leftarrow e$ 
18:  end if
19:   $k \leftarrow k + 1$ 
20: end while
21: return  $g$ 

```

3 Implementation and protocol

3.1 Description of the protocol

This section will describe the numerical protocol that we apply to compare efficiently and in a fair play way all our methods to solve the PFSP problem. First, we will describe the environment and the hardware used 3.1.1. Then, we will precise the stop condition that we used 3.1.2 and how we measured computation time 3.1.3.

3.1.1 Hardware description and environment

Computer used to run the tests All numerical experiments were executed locally on a single machine to ensure a fair and consistent evaluation environment. The hardware configuration of the computer used for these tests is detailed as follows:



Component	Specifications
Processor (CPU)	Intel® Core™ i5-1035G1 @ 1.00 GHz (Base: 1.20 GHz). 4 Physical Cores, 8 Logical Processors. 6.0 MB L3 Cache.
Memory (RAM)	12.0 GB SODIMM operating at 3200 MT/s (Dual-slot configuration).
Operating System	Ubuntu

Table 2: Hardware specifications of the machine used for computation.

Furthermore, to maintain the integrity of the computation time measurements, background tasks were minimized during the execution of the scripts.

Environment and resources allocation UBUNTU allows you to allocate a certain number of cores to a specific task, that's why we allocate a specific core for every experiment run. To do so, we use the following:

```
1 # force the script to run only on core 1
2 taskset -c 1 python app.main
```

Listing 1: Allocating core 1 to run the experiment

3.1.2 Stop condition

To ensure a strictly fair and unbiased comparison between the different stochastic algorithms (Random Search, Evolutionary Algorithm, and Simulated Annealing), we established a common computational budget rather than relying on a time-based limit. The stop condition was strictly defined by a maximum Number of Function Evaluations (NFE) set to 10,000.

The only exception to this rule is the Greedy algorithm. Being a deterministic constructive heuristic rather than a stochastic search method, it does not iterate over complete solutions. Instead, it naturally terminates as soon as the single optimal local schedule is fully built, resulting in an NFE of exactly 1.

3.1.3 Measuring computation time

It is also useful to accurately measure the real-world execution time in order to evaluate the practical efficiency of each algorithm.

To measure the elapsed time in our Python implementation, we use the `timeit` module instead of the most common `time` as `timeit` is much more accurate [8]. As widely documented in Python benchmarking standards, `time.time()` measures the real-world wall-clock time and is susceptible to system clock updates, which can lead to inaccuracies during very short executions.

3.2 Main structure

According to the pseudo-code presented in 2, we have implemented these methods using the Object-oriented programming and the structure is therefore represented through a class diagram **Figure 5**.

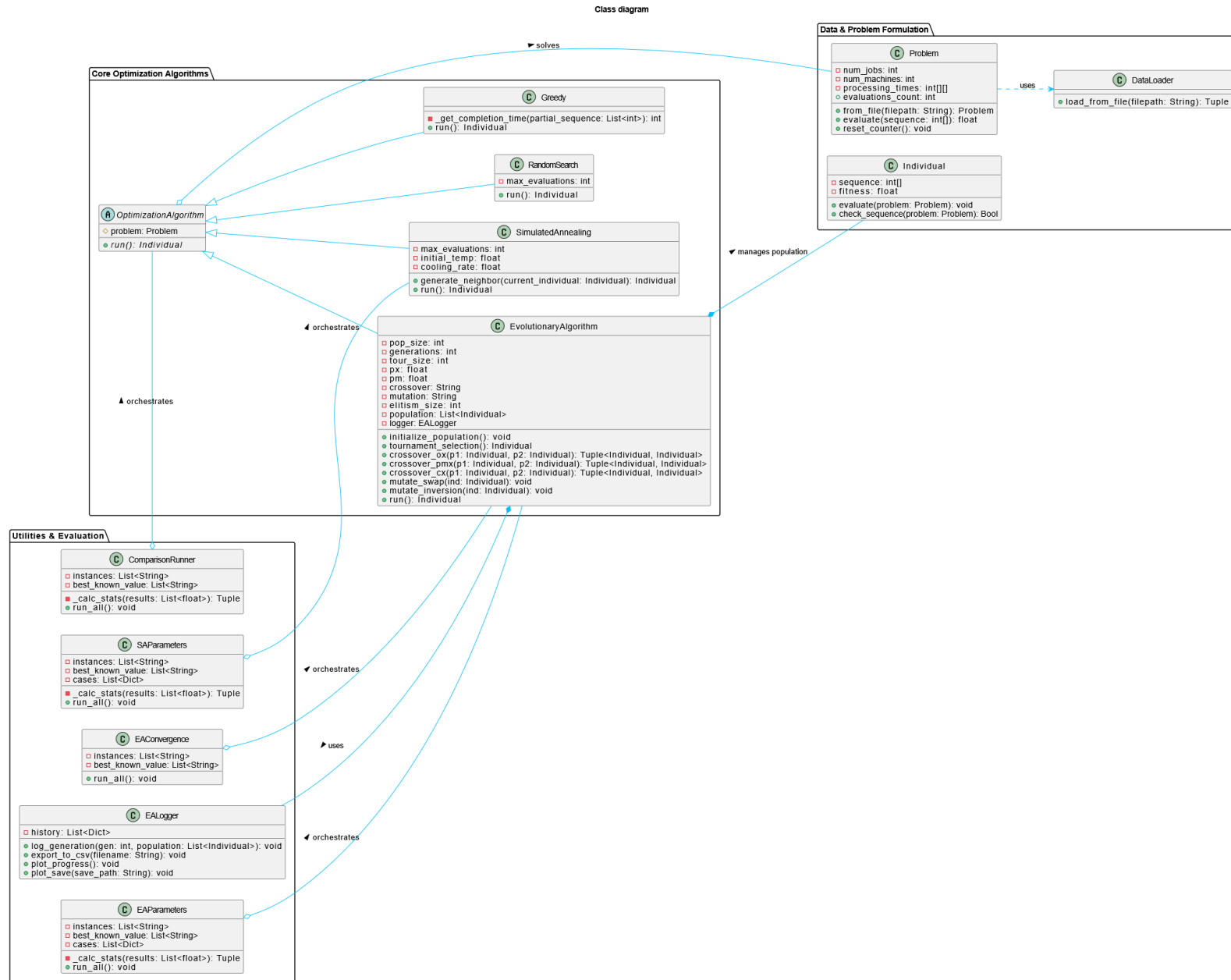


Figure 5: Class diagram of the application implemented.

3.3 Unit tests

Before running all numerical experiments, we implemented unit tests in order to verify critical method or method of algorithm such as the example [Listing 2](#).

```
from app.Problem.Problem import Problem

# TEST

if __name__ == "__main__":
    M = [[30, 29, 19, 50],
          [39, 69, 6, 73],
          [3, 86, 58, 60],
          [72, 74, 51, 15],
          [24, 47, 22, 90]]

    p = Problem(4,5,M)

    print("--- Display test ---")
    print(p)

    print("--- Test on known sequence ---")
    result = p.evaluate([0, 2, 3, 1])
    if result == 1170:
        print(f'Unit test successful, for the sequence [1, 3, 4, 2], we get {result}')
    else:
        print(f"Unit test failed, for the sequence [1, 3, 4, 2], we get {result}')
```

Listing 2: Example of unit test for the evaluation function of a given problem

4 Finding the optimal parameters

As described in [Table 1](#), we will use two methods that require parameters : the Evolutionary Algorithm (EA) and the Simulated Annealing (SA). To compare all these methods, we need to find first the optimal parameters for EA and SA.

First, we present all the cases used to determine the best set of parameters for these two methods 4.1. Then, we run experiment for each case 4.2 and we summarize all the best set of parameters in 4.3.

4.1 Description of the different cases

We will first present the cases in order to determine the best set of parameters for the evolutionary algorithm : the population/generation ratio, crossover probability, mutation probability, tour size, mutation type, crossover type, elitism size 4.1.1.

Then, we will present the cases for the Simulated Annealing including initial temperature and cooling rate parameters 4.1.2.



4.1.1 Evolutionary Algorithm

First, we explore the impact of changing the ratio population/generation size. But, in order to run fair test, we fix the total number of iteration, which is the product of the population and the generation size (here $100 \times 100 = 10\,000$). All other settings remain at their default values.

These cases (Table 3) are designed to try to find a trade-off between, on the one-hand *exploration* (large population and few generation) and *exploitation* (small population and many generations).

As mentioned in the protocol (METTRE REF), we also use average our results over 10 runs.

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
1	100	100	0.7	0.1	5	Swap	OX	0	10
2	20	500	0.7	0.1	5	Swap	OX	0	10
3	1000	10	0.7	0.1	5	Swap	OX	0	10
4	10000	1	0.7	0.1	5	Swap	OX	0	10

Table 3: Description of the parameters of the cases used to determine the best population/generation ratio.

We assume then that we get the best ratio between population and generation size, denoted by a^* and b^* , respectively. Thus, we can study the impact of the crossover probability on the score (Table 4). We also include inside medium condition ($P_x = 0.3$) boundary conditions such as $P_x = 0$ (no crossover) and forced crossover $P_x = 1$.

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
5	a^*	b^*	0.0	0.1	5	Swap	OX	0	10
6	a^*	b^*	0.3	0.1	5	Swap	OX	0	10
7	a^*	b^*	0.7	0.1	5	Swap	OX	0	10
8	a^*	b^*	1.0	0.1	5	Swap	OX	0	10

Table 4: Description of the parameters of the cases used to determine the best crossover probability.

We note c^* the best crossover probability found and we now focus our analysis to the mutation probability P_m . In the same way than the previous parameter, we explore boundary conditions ($P_m = 0$ and $P_m = 1$) as well as a very low rate ($P_m = 0.01$) suitable for swap operation [2] and the default value ($P_m = 0.1$).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
9	a^*	b^*	c^*	0.0	5	Swap	OX	0	10
10	a^*	b^*	c^*	0.01	5	Swap	OX	0	10
11	a^*	b^*	c^*	0.1	5	Swap	OX	0	10
12	a^*	b^*	c^*	1.0	5	Swap	OX	0	10

Table 5: Description of the parameters of the cases used to determine the best mutation probability.

We now note d^* the best mutation probability and we continue our exploration with the size of the tournament (parameter "Tour") (Table 6). We use the default value (5) and as this parameter describe the selection pressure, we explore a bigger value (20) and the boundary conditions : no selection pressure (1) and maximum pressure with the tournament size equal to the population size (a^*).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
13	a^*	b^*	c^*	d^*	1	Swap	OX	0	10
14	a^*	b^*	c^*	d^*	5	Swap	OX	0	10
15	a^*	b^*	c^*	d^*	20	Swap	OX	0	10
16	a^*	b^*	c^*	d^*	a^*	Swap	OX	0	10

Table 6: Description of the parameters of the cases used to determine the best tour size.

Let's assume that we get the best tournament size, that we note e^* . We now compare the scores obtained with the two types of mutation : swap and inversion (Table 7).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
17	a^*	b^*	c^*	d^*	e^*	Swap	OX	0	10
18	a^*	b^*	c^*	d^*	e^*	Inversion	OX	0	10

Table 7: Description of the parameters of the cases used to determine the best mutation type. Comparison between swap and inversion mutation.

We then do the same approach with the crossover types: Ordered Crossover (OX), Partially Matched Crossover (PMX) and the Cycle Crossover (CX) (Table 8).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
19	a^*	b^*	c^*	d^*	e^*	f^*	OX	0	10
20	a^*	b^*	c^*	d^*	e^*	f^*	PMX	0	10
21	a^*	b^*	c^*	d^*	e^*	f^*	CX	0	10

Table 8: Description of the parameters of the cases used to determine the best crossover type.

Finally, we study the impact of the presence of elitism which was absent for all our first cases. Therefore, we compare the situation without elitism (0), and with just one and finally five (Table 9).

Case n°	Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
22	a^*	b^*	c^*	d^*	e^*	f^*	g^*	0	10
23	a^*	b^*	c^*	d^*	e^*	f^*	g^*	1	10
24	a^*	b^*	c^*	d^*	e^*	f^*	g^*	5	10

Table 9: Description of the parameters of the cases used to determine the impact of elitism.

4.1.2 Simulated Annealing

We now present the same approach for the simulated annealing method with only two parameters : the initial temperature and the cooling rate. We use common value (METTRE REF) and try slightly different Initial Temperature (Table 10) in order to find an equilibrium between early exploration and convergence speed.

Case n°	Initial Temperature	Cooling Rate	Run per algo
25	100	0.99	10
26	1000	0.99	10
27	10000	0.99	10

Table 10: Description of the parameters used to determine the best initial temperature for Simulated Annealing.

Let's assume that we have the best initial temperature that we note T^* . We then run the method with a cooling rate closer to one (Table 11).

Case n°	Initial Temperature	Cooling Rate	Run per algo
28	T^*	0.90	10
29	T^*	0.99	10
30	T^*	0.999	10
31	T^*	0.9999	10

Table 11: Description of the parameters used to determine the best cooling rate for Simulated Annealing.

4.2 Running experiment

We will start by finding an optimal set of parameters for the evolutionary algorithm 4.2.1 and then for the simulated annealing 4.2.2.

For every table of results : each instance is described by its number of tasks and machines as $J \times M$; the color green is used to highlight the best score and the best average score for each instance ; Evolutionary Alg. [10x] indicates ten iterations of the evolutionary algorithm and best* (best* indicates the best value from each 10 runs), worst, average and standard deviation (std), computation time and Number of Function Evaluations (NFE) are provided for each case and finally the best known value is indicated.

For each parameters, we present the results from each instances of the same level of difficulty in a separate table.

4.2.1 Evolutionary Algorithm

Best Ratio generation/population size We start by determining the best ratio generation/population size. We present in different table the result from instances of different difficulties: the basic instances in Table 12, the medium instance in Table 13 and the hard instance in Table 14.

For the basic instance, we can see that the best case is the number one as it has the best average score for every cases. The best score for the instance 2 is obtained with case 2 but is very close to the case 1 and, as the process uses randomness, this difference is not relevant. We also can see that all cases have the same computation time but the case 4 is slightly faster. This case 4 has indeed a huge population size but only one generation, it is therefore like a random search which decrease the average score but decrease also the computation time.

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	1	2	3	4	1	2	3	4	1	2	3	4
Score	best*	14222.0	14254.0	14285.0	15352.0	21102.0	21094.0	21417.0	22829.0	33963.0	34113.0	34172.0	35670.0
	worst	14647.0	14569.0	14605.0	15628.0	22057.0	22304.0	21935.0	23243.0	34517.0	35238.0	34813.0	36238.0
	avg	14366.2	14374.9	14410.7	15482.9	21689.0	21742.2	21725.9	23075.6	34304.3	34642.4	34585.7	36036.2
	std	125.35	106.58	99.63	81.98	314.25	341.53	196.77	130.07	182.33	353.26	191.34	187.23
Time	second (s)	10.887	10.731	11.085	8.956	19.405	19.209	19.055	17.305	36.425	36.045	36.032	33.833
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 12: Results for each case and each basic instance to get the best ratio generation/population size for the evolutionary algorithm.

We get the same results for the medium instances with the best score and the best average value for the case 1 and the worst for the case 4 but with a smaller computation time.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	1	2	3	4	1	2	3	4
Score	best*	324127.0	324276.0	342726.0	358835.0	398937.0	402332.0	416916.0	435885.0
	worst	331236.0	335865.0	346552.0	364417.0	408207.0	411199.0	424099.0	441092.0
	avg	327296.7	330382.8	344602.9	361473.6	404436.0	406637.8	421884.1	439011.4
	std	2411.51	3262.17	1477.53	1905.43	2741.5	3743.87	2058.67	1452.86
Time	second (s)	93.392	92.168	90.77	83.388	179.086	183.177	177.992	169.066
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 13: Results for each case and each medium instance to get the best ratio generation/population size for the evolutionary algorithm.

Regarding the hard case, the worst case is still the case 4 but the case with the best average value is now case 2. Indeed, this case with a small population (20) but a lot of generations (500) has a bigger exploration space than the case 1 with only 100 generations. Therefore, as the search space of the hard instance is bigger, the case 2 is better. However, we can notice that the difference between the average value of case 1 and 2 is only 0.016%.

Finally, we can conclude that the case 1 is the best overall (except for the hard instance but the difference is very minor). Therefore, we adopt the parameters of 100 individuals in the population and 100 generations, so $a^* = 100$ and $b^* = 100$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	1	2	3	4
Score	best*	7500875.0	7490765.0	7746411.0	7888137.0
	worst	7608577.0	7576741.0	7786773.0	7943570.0
	avg	7536436.2	7535204.7	7765615.4	7915677.5
	std	33531.22	28554.9	12073.46	19238.63
Time	second (s)	965.183	970.792	959.711	839.728
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 14: Results for each case on the hard instance to get the best ratio generation/population size for the evolutionary algorithm.

Best Crossover probability Once we have $a^* = 100$ and $b^* = 100$, we can proceed the case 5 to 8 in order to determine the best crossover probability.

For the basic instances, we can see that the case with the best average value is always the case 8 and the worst (on average) is the case 5 (Table 15). Indeed the case 5 corresponds to a probability $P_x = 0$ so there is no more crossover. Therefore, the method just copy individuals with just a mutation so the algorithm loses its capability to explore a huge space domain and the algorithm is just exploring randomly in the local search space. Thus, the mutation alone is not able to converge the evolutionary algorithm. We also observe that the best score for instance 2 and 8 are obtained with the case 7 but the value is very close to the case 8 and, as the method is random, the difference is not relevant.

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	5	6	7	8	5	6	7	8	5	6	7	8
Score	best*	14183.0	14198.0	14130.0	14096.0	21419.0	21412.0	21001.0	21030.0	34009.0	34127.0	33830.0	33983.0
	worst	14668.0	14559.0	14387.0	14406.0	22395.0	22010.0	21952.0	21747.0	35228.0	34665.0	34789.0	34899.0
	avg	14378.2	14348.6	14270.2	14197.8	21925.9	21775.4	21513.1	21350.6	34644.5	34434.9	34510.0	34282.4
	std	171.85	100.73	86.03	91.4	334.96	185.14	268.22	204.99	316.67	162.93	289.64	264.75
Time	second (s)	10.772	10.724	10.639	10.683	19.758	19.641	19.197	18.985	35.865	35.823	35.749	35.7
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 15: Results for each case and each basic instance to get the best crossover probability for the evolutionary algorithm.

We observe the same trend for the medium instances with the worst average case (5) and the best average case (8) (Table 16). The case 8 correspond to a crossover probability equal to 1 so there is a crossover at each step so the evolutionary algorithm is constantly changing. As the other default parameters are a good pressure selection with a tour size of 5 and an OX-crossover, this crossover probability helps the algorithm to find local optimum faster. We can also notice that for this case, the standard deviation is much smaller (46.81% for instance 1, 38.9% for instance 2, 27% for instance 4 for example) which leads us to conclude that convergence is happening more quickly.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	5	6	7	8	5	6	7	8
Score	best*	331935.0	326407.0	326543.0	322743.0	408184.0	405759.0	401258.0	397746.0
	worst	346741.0	336932.0	334801.0	333509.0	417669.0	413777.0	408353.0	406913.0
	avg	338030.3	332789.2	331122.8	327199.8	413156.8	408785.9	405320.2	401610.5
	std	4214.69	3430.35	2507.97	3076.81	2854.14	2682.9	2583.73	3054.56
Time	second (s)	89.903	90.907	92.264	93.806	173.785	176.184	177.271	180.553
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 16: Results for each case and each medium instance to get the best crossover probability for the evolutionary algorithm.

Regarding the hard instance, we get the same result and a standard deviation for the case 8 35.17% smaller than the case 5. We can also observe that the computation is bigger for case 8 than case 5 as it takes time to run the crossover (there is no more crossover for case 5).

Finally, we can conclude that the best overall case is the case 8 with a crossover probability of $P_x = 1$ so we get $c^* = 1$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	5	6	7	8
Score	best*	7633865.0	7558436.0	7495775.0	7440933.0
	worst	7753107.0	7633119.0	7559650.0	7509208.0
	avg	7706413.5	7585329.5	7518667.2	7476693.9
	std	32814.73	27160.69	19751.69	21272.84
Time	second (s)	856.528	910.855	986.947	1035.307
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 17: Results for each case on the hard instance to get the best crossover probability for the evolutionary algorithm.

Best Mutation Probability As we get $c^* = 1$, we can now run the experiments to determine the best mutation probability (cases 9 to 12).

We can see that the best average value is for the case 12 for the basic instances (Table 18), which correspond to a mutation probability of one. This result is interesting because it should mean that all the sequence is mutated so the information is destroyed. However, this is not the case as we have implemented the mutation on the whole individual and not on each genes (see Listing 3). Therefore, even with a mutation probability of 1, we only change an unique swap so this huge probability is in fact beneficial for the exploration.

```

if random.random() < self.pm:
    idx1, idx2 = random.sample(range(len(individual.sequence)), 2)
    individual.sequence[idx1], individual.sequence[idx2] = ...

```

Listing 3: Implementation of the mutation

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	9	10	11	12	9	10	11	12	9	10	11	12
Score	best*	14203.0	14205.0	14095.0	14158.0	21129.0	21228.0	21459.0	21175.0	34198.0	34025.0	34014.0	34127.0
	worst	14943.0	14738.0	14465.0	14448.0	21950.0	21975.0	22071.0	21934.0	35041.0	34675.0	34585.0	34462.0
	avg	14460.3	14382.0	14295.0	14281.1	21555.5	21629.7	21736.1	21476.1	34615.4	34373.9	34334.3	34320.9
	std	219.84	163.85	120.92	88.7	275.18	230.69	220.81	211.97	261.74	239.99	174.83	142.37
Time	second (s)	10.731	10.643	10.634	11.124	20.046	19.258	19.038	19.182	35.638	35.658	35.682	35.936
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 18: Results for each case and each basic instance to get the best mutation probability for the evolutionary algorithm.

We observe the same results for the medium instances Table 19, the worst case is still the case 9 where the mutation probability is 0. In this case, indeed, there is no more mutation so we cannot create again a lost gene. Therefore, the algorithm is converging prematurely and that proves that mutation is absolutely necessary to maintain enough diversity in the population.

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	9	10	11	12	9	10	11	12
Score	best*	329304.0	326378.0	321181.0	322710.0	407525.0	400084.0	397951.0	398659.0
	worst	341982.0	335778.0	330675.0	330083.0	411538.0	410820.0	406257.0	405660.0
	avg	334502.0	331145.4	326455.5	326618.8	409689.2	406938.4	402739.6	401484.8
	std	4057.82	3223.72	3115.71	2270.6	1218.14	3436.72	2651.99	2604.52
Time	second (s)	93.16	91.567	91.614	91.862	175.273	174.932	175.036	175.408
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 19: Results for each case and each medium instance to get the best mutation probability for the evolutionary algorithm.

We observe the same best and worst average case for the hard instance (Table 20). We finally can notice that results from case 11 ($P_m = 0.1$) and 12 ($P_m = 1$) are very close and the case 11 is even better than case 12 in the instance 4. However, as the case 12 promotes exploration, it is even better when the instance is hard.

We can therefore conclude that the best overall is the case 12 with a mutation probability of $P_m = 1$, thus $d^* = 1$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	9	10	11	12
Score	best*	7478639.0	7528530.0	7413854.0	7320732.0
	worst	7627611.0	7583571.0	7545384.0	7465969.0
	avg	7575187.3	7558861.3	7464309.6	7405428.6
	std	41445.27	21023.83	43909.96	39549.72
Time	second (s)	1060.226	1070.248	1069.685	1070.132
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 20: Results for each case on the hard instance to get the best mutation probability for the evolutionary algorithm.

Best Tour Size As we just get $d^* = 1$, we are now able to run the tests on the tour size (case 13 to 16).

First, we can notice that the overall worst case is the case 13 with tour size of 1 because there is therefore no selection pressure so the parents are chosen without any regard on the quality of the individual. Thus, this case is more likely the random search.

We can observe on the basic instances that the extreme selection of a tour size of 100 (case 16) does not provide as good solution as case 15 (with a tour size of 20) (Table 21).

		Evolutionary Alg. [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	13	14	15	16	13	14	15	16	13	14	15	16
Score	best*	15293.0	14231.0	14054.0	14083.0	22778.0	21320.0	20971.0	21159.0	35672.0	34261.0	33770.0	34040.0
	worst	15608.0	14404.0	14255.0	14373.0	23373.0	22000.0	21581.0	22065.0	36364.0	34606.0	34342.0	34489.0
	avg	15402.3	14289.5	14120.9	14199.7	23119.4	21582.5	21248.4	21596.5	36065.5	34440.7	34119.3	34301.9
	std	122.88	60.49	64.6	93.74	231.07	268.92	183.91	305.57	242.46	106.08	180.64	167.16
Time	second (s)	10.646	10.779	11.602	13.6	19.812	20.343	21.039	22.73	37.671	37.792	38.333	40.113
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 21: Results for each case and each basic instance to get the best tour size for the evolutionary algorithm.

On the medium instances, the worst case is still the case 13 but the best case becomes the case 16 as this pressure is interacting with the mutation probability (Table 22). However, the case 15 provides very close results (0.6% for the instance 4 for example).

		Evolutionary Alg. [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	13	14	15	16	13	14	15	16
Score	best*	360831.0	322772.0	319697.0	315635.0	435493.0	397925.0	391204.0	388737.0
	worst	364669.0	328417.0	323292.0	321219.0	442487.0	404076.0	396854.0	398003.0
	avg	362581.4	325470.8	321665.9	319593.4	439164.3	401260.9	394464.9	393110.1
	std	1313.51	1706.17	1102.11	1717.19	2100.67	2326.47	1641.03	2706.11
Time	second (s)	95.031	97.485	98.495	100.354	186.515	186.394	184.712	184.812
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 22: Results for each case and each medium instance to get the best tour size for the evolutionary algorithm.

On the hard instance (Table 23), the best overall case is still the case with an extreme pressure selection (case 16). As this case is not able to provide good results on the basic instances and as the case 15 provides better solution on these instances and still find close solutions for medium and hard instances, we can conclude that the best overall tour size is 20 (case 15). Moreover, this is more than the default value of 5 because we had already a lot of exploration (as $P_x = 1$ and $P_m = 1$) so we need a pressure selection to counterbalance this huge exploration. Thus, we get $c^* = 20$.

		Evolutionary Alg. [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	13	14	15	16
Score	best*	7865718.0	7377624.0	7320370.0	7316248.0
	worst	7963803.0	7451755.0	7380543.0	7372040.0
	avg	7919021.9	7411693.7	7357453.1	7350482.0
	std	36021.12	25345.13	20160.08	21873.88
Time	second (s)	1045.47	1064.819	1073.462	1074.66
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 23: Results for each case on the hard instance to get the best tour size for the evolutionary algorithm.

Best Mutation Type We will now present the results in order to select either the swap or the inversion mutation. The results of these mutation are very close for the basic instances but the swap is on average better (Table 24). For the instance 3, by chance the inversion get a better best score.

		Evolutionary Alg. [10x]					
		Inst. 1 (20x5)		Inst. 2 (20x10)		Inst. 3 (20x20)	
Data	Case n° Stat.	17	18	17	18	17	18
Score	best*	14059.0	14085.0	21011.0	21012.0	33995.0	33953.0
	worst	14232.0	14222.0	21663.0	22190.0	34354.0	34555.0
	avg	14114.0	14139.2	21317.4	21514.5	34132.3	34243.0
	std	61.48	51.08	209.62	438.97	104.5	228.18
Time	second (s)	11.522	11.625	20.522	21.091	38.49	38.453
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	20911	20911	33623	33623

Table 24: Results for each case and each basic instance to get the best mutation type for the evolutionary algorithm.

The swap mutation is still the best mutation type for the medium instances but we can observe that the difference between these two methods is greater (Table 25).

		Evolutionary Alg. [10x]			
		Instance 4 (100x10)		Instance 5 (100x20)	
Data	Case n° Stat.	17	18	17	18
Score	best*	317347.0	325925.0	391503.0	406077.0
	worst	323744.0	338217.0	399328.0	417778.0
	avg	320827.7	331761.3	395960.8	412763.4
	std	1803.07	3917.07	2911.22	3922.15
Time	second (s)	97.18	97.127	178.997	177.733
NFE	number	10000	10000	10000	10000
Best Known Value		300566	300566	367267	367267

Table 25: Results for each case and each medium instance to get the best mutation type for the evolutionary algorithm.

When it comes to the hard instance (Table 26), the difference is now of approximately 240 000 so the swap is definitively better for the hard instance. Therefore, we can conclude that the best mutation type overall is the swap so we get $f^* = swap$.

		Evolutionary Alg. [10x]	
		Instance 6 (500x20)	
Data	Case n° Stat.	17	18
Score	best*	7315598.0	7529857.0
	worst	7388985.0	7656086.0
	avg	7355471.4	7596958.2
	std	22598.95	37357.41
Time	second (s)	1048.25	1047.768
NFE	number	10000	10000
Best Known Value		6697476	6697476

Table 26: Results for each case on the hard instance to get the best mutation type for the evolutionary algorithm.

Best Crossover Type We will now present the results for the choice of the crossover type.

For the basic instances, the crossover is the best mutation on average but all mutation type have similar value (Table 27). We can notice that we reach the best known value with the OX mutation for the instance 3.

		Evolutionary Alg. [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	19	20	21	19	20	21	19	20	21
Score	best*	14062.0	14046.0	14034.0	21023.0	20958.0	21117.0	33623.0	33812.0	33887.0
	worst	14251.0	14287.0	14258.0	21609.0	21884.0	21686.0	34453.0	34381.0	34415.0
	avg	14132.7	14160.6	14146.3	21228.9	21325.7	21399.6	34036.5	34126.5	34097.0
	std	69.68	70.15	74.73	184.67	243.52	183.82	211.52	178.98	161.31
Time	second (s)	11.409	11.171	10.835	20.728	20.675	19.473	36.724	37.309	36.87
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 27: Results for each case and each basic instance to get the best crossover type for the evolutionary algorithm.

On medium instances, we can observe that the CX mutation is the worst (Table 28). This mutation preserve indeed the absolute position of an element but not the relative position. However, in the PFSP, it is the relative position that matters the most and that is the reason why the CS mutation is not optimal for this problem.

		Evolutionary Alg. [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	19	20	21	19	20	21
Score	best*	316081.0	314947.0	318773.0	389043.0	390247.0	393423.0
	worst	323333.0	322210.0	323968.0	399255.0	399593.0	398363.0
	avg	320083.2	318714.1	321398.7	394244.6	394376.4	396291.5
	std	2140.83	2103.99	1675.99	2613.91	2868.01	1556.02
Time	second (s)	94.265	88.484	87.536	180.122	173.232	172.941
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	367267	367267	367267

Table 28: Results for each case and each medium instance to get the best crossover type for the evolutionary algorithm.

On the medium instances and on the hard instance, we can observe that both OX and PMX obtained similar good results. However, we can see that the PMX is much faster than the OX, especially on the hard instance (Table 28) with a difference of 16.96% for the instance 6. That is why, even if both crossover have the same results (OX slightly better for the medium and basics instances and PMX slightly better for the hard instance), we can finally conclude that the PMX crossover is the best overall. Thus, we have $g^* = \text{PMX}$.

		Evolutionary Alg. [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	19	20	21
Score	best*	7326669.0	7295784.0	7351373.0
	worst	7394134.0	7347446.0	7449460.0
	avg	7352788.6	7323820.3	7397009.3
	std	19627.19	15456.29	30614.76
Time	second (s)	1038.401	868.282	853.303
NFE	number	10000	10000	10000
Best Known Value		6697476	6697476	6697476

Table 29: Results for each case on the hard instance to get the best crossover type for the evolutionary algorithm.

Impact of Elitism We have now all our final parameters and we can now study the impact of elitism. First, we can observe that the NFE for the case 24 (elitism of 5) is not 10 000 but 9 604 and this difference comes from our implementation of elitism (Table 30). As we copy 5 best individuals from the previous generation, we just have to create 95 new children to fill our generation so we loop 48 times (because we create children by pairs so $48 \times 2 = 96$ so we have in total $100 + (99 \times 96) = 9604$ evaluations in total.

Furthermore, we can observe that the case without elitism (case 22) is the best on average for the basic

instances.

		Evolutionary Alg. [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	22	23	24	22	23	24	22	23	24
Score	best*	14066.0	14060.0	14041.0	20955.0	20981.0	21081.0	33897.0	33939.0	33983.0
	worst	14247.0	14334.0	14323.0	21769.0	21979.0	21879.0	34413.0	34430.0	34544.0
	avg	14137.9	14159.8	14189.0	21226.4	21447.5	21401.8	34162.5	34218.3	34210.6
	std	56.79	87.22	85.91	211.48	348.45	262.52	151.68	155.91	168.04
Time	second (s)	11.132	11.12	10.805	20.359	20.595	18.969	36.431	36.907	35.28
NFE	number	10000	10000	9604	10000	10000	9604	10000	10000	9604
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 30: Results for each case and each basic instance to study the impact of elitism for the evolutionary algorithm.

Regarding the medium instances, the case without elitism and with a little bit of elitism (case 23) are very close but the case with a lot of elitism (case 24) is the worst (Table 31).

		Evolutionary Alg. [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	22	23	24	22	23	24
Score	best*	314503.0	315850.0	315638.0	391221.0	389343.0	389667.0
	worst	322209.0	321540.0	324126.0	396354.0	396602.0	396029.0
	avg	318695.9	318630.6	319206.7	393746.6	393634.2	392964.9
	std	2122.58	2036.45	2940.23	1883.06	2415.28	2477.69
Time	second (s)	89.431	88.455	85.575	175.416	175.712	167.002
NFE	number	10000	10000	9604	10000	10000	9604
Best Known Value		300566	300566	300566	367267	367267	367267

Table 31: Results for each case and each medium instance to study the impact of elitism for the evolutionary algorithm.

When it comes to the hard instance, the case without elitism is the best and the difference with the other cases is bigger than the medium instances. As we have a very dynamic algorithm with $P_m = 1$ and $P_x = 1$, the elitism will keep the best solution and slow the exploration dynamic so the algorithm will converge prematurely.

Before concluding, we must question the fairness of the results as the NFE is not the same for the three cases. First, we need to notice that there is just a difference of 4% between 10 000 and 9 604 so it is very likely to not change the results (especially on the instance 6 where the difference is big) because the difference is equivalent to just 4 generations, which is not enough to change significantly the results.

Thus, we can conclude that the best parameter overall is no elitism so $h^* = 0$.

		Evolutionary Alg. [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	22	23	24
Score	best*	7303082.0	7320097.0	7323373.0
	worst	7358827.0	7384673.0	7376432.0
	avg	7329992.0	7347818.3	7353550.7
	std	19016.22	21781.09	20212.65
Time	second (s)	851.547	854.795	827.366
NFE	number	10000	10000	9604
Best Known Value		6697476	6697476	6697476

Table 32: Results for each case on the hard instance to study the impact of elitism for the evolutionary algorithm.

4.2.2 Simulated Annealing

Best Initial Temperature We will now present our results to determine the best initial temperature from 100 and 1000 to 10000. All these temperatures provide very close results on the basic instances [Table 33](#), the medium instances [Table 34](#) and the hard instance [Table 35](#).

However, we can observe on these tables that the case 26 (so an initial temperature of 1000) is the best on average on the instances 1, 3, 4 and 6 (the most difficult) and, moreover, that the standard deviation is lower on the instances 3, 4 and 6, which led us to conclude that this temperature is slightly better for our problem.

The case 25 correspond indeed to an initial temperature of 100 lower than 1000 so the algorithm will refuse faster the false solution and the case 27 correspond to 10000 which is better for the exploration phase but this phase is therefore longer and the exploitation is not enough, especially in the instance 6.

Thus, we can conclude that the initial temperature of 1000 is the best overall and we have therefore $T^* = 1000$.

		Simulated Annealing [10x]								
		Inst. 1 (20x5)			Inst. 2 (20x10)			Inst. 3 (20x20)		
Data	Case n° Stat.	25	26	27	25	26	27	25	26	27
Score	best*	14129.0	14211.0	14080.0	21189.0	21434.0	21291.0	34096.0	34088.0	34138.0
	worst	14644.0	14544.0	14697.0	22335.0	22252.0	22253.0	35603.0	34882.0	35071.0
	avg	14363.9	14331.3	14345.4	21792.7	21806.5	21773.5	34787.1	34436.0	34630.3
	std	167.81	101.78	167.89	401.44	325.76	308.15	474.31	270.61	297.21
Time	second (s)	8.504	8.462	8.536	17.052	17.628	17.708	34.079	33.488	34.009
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	20911	20911	20911	33623	33623	33623

Table 33: Results for each case and each basic instance to get the best initial temperature for the simulated annealing.



		Simulated Annealing [10x]					
		Instance 4 (100x10)			Instance 5 (100x20)		
Data	Case n° Stat.	25	26	27	25	26	27
Score	best*	313684.0	313770.0	313280.0	387497.0	387210.0	386715.0
	worst	318940.0	316809.0	319616.0	392393.0	394287.0	393934.0
	avg	316996.6	315332.2	315959.9	389913.2	390828.7	389639.7
	std	1573.81	1082.29	1769.95	1497.3	2714.8	2760.49
Time	second (s)	83.407	84.381	84.224	170.249	170.942	171.045
NFE	number	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	367267	367267	367267

Table 34: Results for each case and each medium instance to get the best initial temperature for the simulated annealing.

		Simulated Annealing [10x]		
		Instance 6 (500x20)		
Data	Case n° Stat.	25	26	27
Score	best*	7153416.0	7178187.0	7162626.0
	worst	7253803.0	7241575.0	7270542.0
	avg	7207245.8	7206331.1	7217178.8
	std	32178.82	20538.91	34666.81
Time	second (s)	838.654	840.779	837.955
NFE	number	10000	10000	10000
Best Known Value		6697476	6697476	6697476

Table 35: Results for each case on the hard instance to get the best initial temperature for the simulated annealing.

Best Cooling Rate As presented in 2.5, equation (8), the current temperature is given by the following:

$$T = \lambda^{\text{NFE}} T_0 \quad (9)$$

Let's compare this temperature for the different cooling rate λ . If $\lambda = 0.9$ (case 28), the temperature is near 0 after a few hundred iterations so the algorithm will process a random search after a few hundred iterations which explain why the results of this case is average.

If $\lambda = 0.99$ (case 29), the temperature is also near 0 very fast but not as fast as in the case 28 so this case is a compromise between exploitation and exploration and this is why this case provide very good results with the instance 6 (very close to the best case on average).

If $\lambda = 0.999$ (case 30), the final temperature is 0.045 which is just near zero so the exploitation and exploration is finished just after 10 000 evaluations.



If $\lambda = 0.9999$ (case 31), then the final temperature is around 368 so the algorithm still accept worse solution so the exploitation is not yet finished.

		Simulated Annealing [10x]											
		Inst. 1 (20x5)				Inst. 2 (20x10)				Inst. 3 (20x20)			
Data	Case n° Stat.	28	29	30	31	28	29	30	31	28	29	30	31
Score	best*	14217.0	14208.0	14049.0	14738.0	21299.0	21117.0	21208.0	21897.0	34073.0	34210.0	34007.0	34946.0
	worst	14695.0	14665.0	14345.0	15154.0	22252.0	22585.0	21980.0	22762.0	34816.0	34728.0	34631.0	35627.0
	avg	14376.1	14383.9	14208.3	14941.9	21694.5	21768.7	21481.3	22370.3	34508.2	34475.4	34251.9	35265.7
	std	160.95	134.78	101.67	140.61	317.73	475.87	287.93	264.43	211.71	205.53	162.76	183.97
Time	second (s)	8.425	8.436	8.436	8.652	17.802	17.621	17.504	17.066	33.683	33.699	33.879	34.127
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		14033	14033	14033	14033	20911	20911	20911	20911	33623	33623	33623	33623

Table 36: Results for each case and each basic instance to get the best cooling rate for the simulated annealing.

Furthermore, we can observe on the basic instances [Table 36](#) and the medium instances [Table 37](#) that the case 30 is the best on average for instance 1, 2, 3 and 4.

		Simulated Annealing [10x]							
		Instance 4 (100x10)				Instance 5 (100x20)			
Data	Case n° Stat.	28	29	30	31	28	29	30	31
Score	best*	315515.0	311141.0	313203.0	324766.0	386730.0	387407.0	388299.0	395165.0
	worst	319258.0	320268.0	316507.0	330087.0	392984.0	394815.0	392236.0	402408.0
	avg	316743.4	315174.1	315110.5	328330.3	389859.9	390010.8	390305.8	399671.0
	std	1327.58	2533.88	1269.85	1544.48	1914.81	2520.44	1404.08	2079.87
Time	second (s)	83.402	83.018	84.306	83.741	169.287	171.097	171.812	172.504
NFE	number	10000	10000	10000	10000	10000	10000	10000	10000
Best Known Value		300566	300566	300566	300566	367267	367267	367267	367267

Table 37: Results for each case and each medium instance to get the best cooling rate for the simulated annealing.

Lastly, on the hard instance ([Table 38](#)), the case 30 is very close to the best one average (case 31) so we can finally conclude that the best overall is the case 30. Thus, we take a cooling rate of $\lambda = 0.999$.

		Simulated Annealing [10x]			
		Instance 6 (500x20)			
Data	Case n° Stat.	28	29	30	31
Score	best*	7191462.0	7164601.0	7183719.0	7148733.0
	worst	7242522.0	7234953.0	7277444.0	7243627.0
	avg	7218795.0	7198379.1	7215244.5	7190556.5
	std	18751.32	18516.45	29218.04	29526.12
Time	second (s)	879.441	882.499	879.332	859.753
NFE	number	10000	10000	10000	10000
Best Known Value		6697476	6697476	6697476	6697476

Table 38: Results for each case on the hard instance to get the best cooling rate for the simulated annealing.

4.3 Summary of the parameters

Population size	Generations size	P_x	P_m	Tour	Mutation Type	Crossover Type	Elitism	Run per algo
100	100	1	1	20	swap	PMX	0	10

Table 39: Final set of parameters for the evolutionary algorithm.

Initial Temperature	Cooling Rate	Run per algo
1000	0.9999	10

Table 40: Final set of parameters for the simulated annealing.

5 Solving the permutation flowshop scheduling problem

5.1 Comparison of all methods on every instances

On basic instances, the evolutionary algorithm is the best method to solve the PFSP. For the first instance, we almost reach the best known value of the problem (in yellow on Table 41). As the search space is relatively small compared to medium or hard instances, the EA approach is well-designed because the 100 generations are able to explore the whole feasible set.

Instance	BKV	Data	Random Alg. [10k]				Greedy Alg.	Evolutionary Alg. [10x]				Simulated Annealing [10x]			
			best*	worst	avg	std	best*	best*	worst	avg	std	best*	worst	avg	std
1 20x5	14033	Score	15214.0	15689.0	15389.2	147.4	16384.0	14034.0	14384.0	14143.3	119.6	14064.0	14433.0	14229.2	101.57
		Time	8.439 s				0.009 s	11.073 s				8.392 s			
		NFE	10000				1	10000				10000			
2 20x10	20911	Score	22947.0	23425.0	23140.8	138.8	25691.0	21126.0	21828.0	21380.2	243.98	21065.0	21852.0	21440.5	242.49
		Time	17.316 s				0.016 s	19.965 s				17.468 s			
		NFE	10000				1	10000				10000			
3 20x20	33623	Score	35362.0	36366.0	35989.1	295.15	35670.0	33844.0	34351.0	34116.2	147.58	34114.0	34544.0	34309.3	164.06
		Time	35.133 s				0.029 s	38.23 s				35.278 s			
		NFE	10000				1	10000				10000			

Table 41: Comparison of all methods to solve the PFSP on basic instances. On average, the evolutionary algorithm is better and the worst is the greedy algorithm.

Regarding the medium instances, the simulated annealing become better than the evolutionary algorithm (Table 42). The search space become indeed much bigger (more than 100 jobs) so the simulated annealing have more time to explore and exploit than the evolutionary algorithm.

We also can observe that the greedy algorithm is the fastest algorithm but provide the worst solution.

Inst.	BKV	Data	Random Alg. [10k]				Greedy Alg.	Evolutionary Alg. [10x]				Simulated Annealing [10x]			
			best*	worst	avg	std	best*	best*	worst	avg	std	best*	worst	avg	std
4 100x10	300566	Score	358758.0	362948.0	360966.9	1323.05	378986.0	316243.0	321988.0	318232.4	1547.14	313350.0	316995.0	315170.6	1292.03
		Time	84.417 s				1.47 s	91.079 s				85.692 s			
		NFE	10000				1	10000				10000			
5 100x20	367267	Score	435112.0	440283.0	438221.0	1872.27	452264.0	390586.0	399575.0	395077.0	2542.83	386983.0	391537.0	388893.2	1505.67
		Time	173.95 s				2.97 s	179.972 s				175.117 s			
		NFE	10000				1	10000				10000			

Table 42: Comparison of all methods to solve the PFSP on medium instances. On average, the simulated annealing is better and the worst is the greedy algorithm.

For the hard instance, as for the medium instances, the simulated annealing is better than the evolutionary algorithm and the difference between the greedy algorithm and the random search is much bigger (Table 43). The random search is indeed much more effective than the greedy algorithm. By making at each step the best solution, the greedy algorithm is not able to reach the "hills" of global maximum. However, the greedy algorithm is faster than the random search.

Inst.	BKV	Data	Random Alg. [10k]				Greedy Alg.	Evolutionary Alg. [10x]				Simulated Annealing [10x]			
			best*	worst	avg	std	best*	best*	worst	avg	std	best*	worst	avg	std
6 500x20	6697476	Score	7890881.0	7944291.0	7917815.3	19220.59	8521746.0	7275153.0	7359946.0	7320114.0	26910.31	7165879.0	7251484.0	7217846.1	25444.85
		Time	884.357 s				363.454 s	885.266 s				873.359 s			
		NFE	10000				1	10000				10000			

Table 43: Comparison of all methods to solve the PFSP on the hard instance. On average, the simulated annealing is better and the worst is the greedy algorithm.

5.2 Convergence of Evolutionary Algorithm

Using the final parameters Table 39, we can study the convergence across the generations of the evolutionary algorithm.

On basic instances, the convergence is very rapid, the steady state is reached after less than 100 generations. Once this global optimum found, the best and average solution is almost constant. However, the



worst solution is higher and has more fluctuations because we use swap with a probability of 1 so there is always perturbation in order to preserve the diversity within the population (Figure 6).

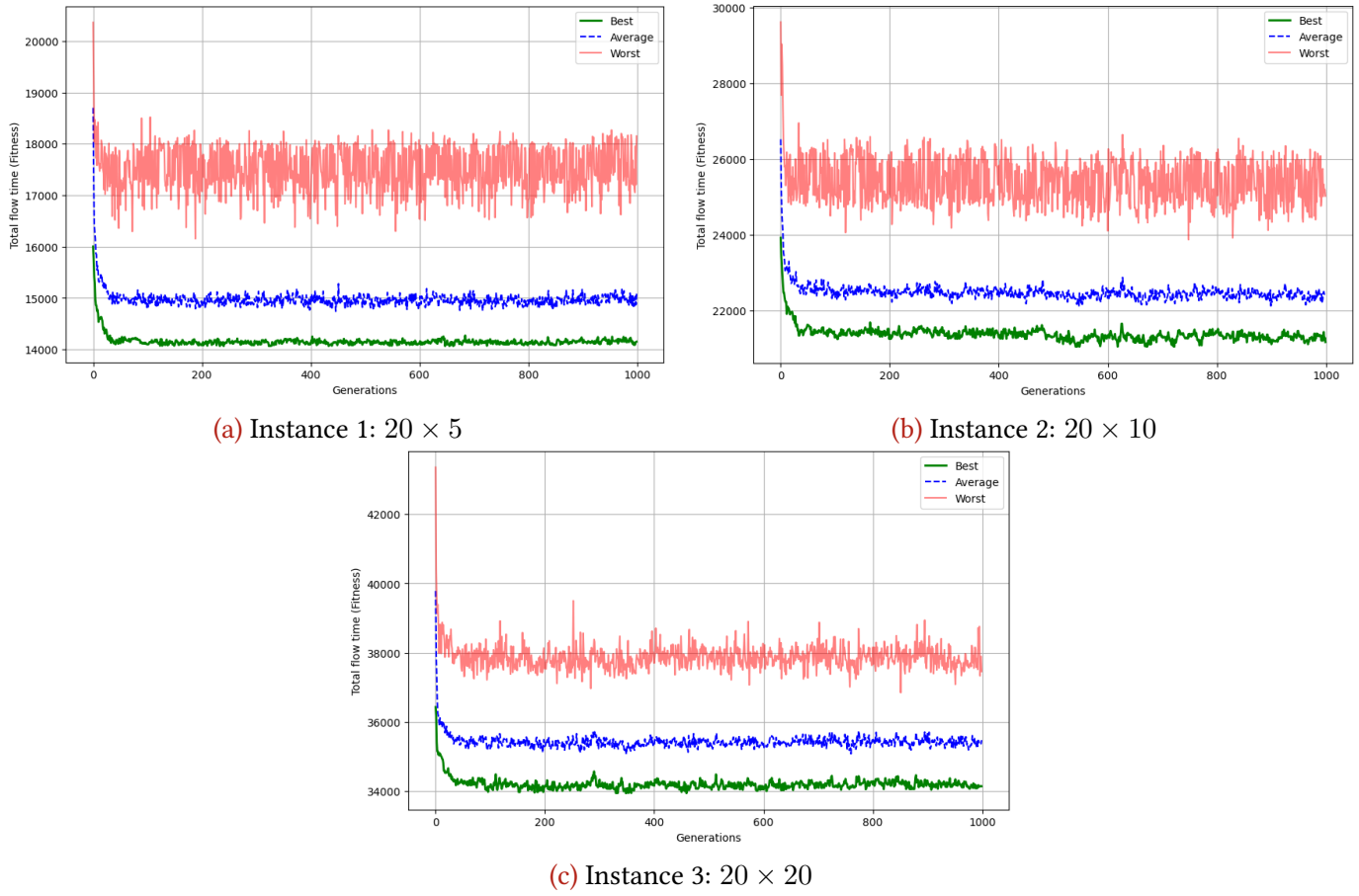


Figure 6: Evolution of the objective function across generations for basic instances.

On medium instances, the convergence takes more time: we need between 200 and 400 generations to find the global optimum. Even after, we observe that the method is still improving the solution, especially on the instance 5 (Figure 7). However, the red line (the worst solution) continues to fluctuate, but the relative deviation from the average is narrowing because the percentage of swap on a bigger generation is smaller so there is less perturbation.

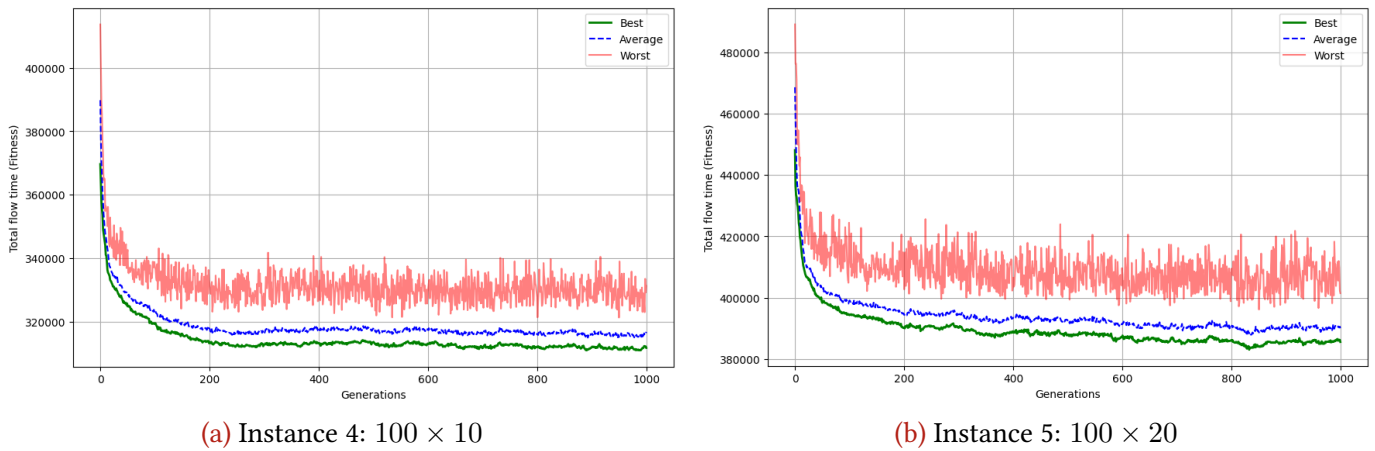


Figure 7: Evolution of the objective function across generations for medium instances.

With the hard instance, there is fewer fluctuation of the worst solution and the convergence is much slower but the solution is constantly improving : the best, the average and the worst (Figure 8). We can see that 1000 generations are not enough to converge, which can explain why the simulated annealing is better than evolutionary algorithm for hard instance.

These graphs show that the evolutionary algorithm exhibits excellent initial convergence dynamics, but that the nature of the mutation (swap) changes radically depending on the size of the instance. It shifts from being a highly exploratory and disruptive operator on small problems (Figure 6) to a very refined local exploitation operator on large-scale problems (Figure 8).

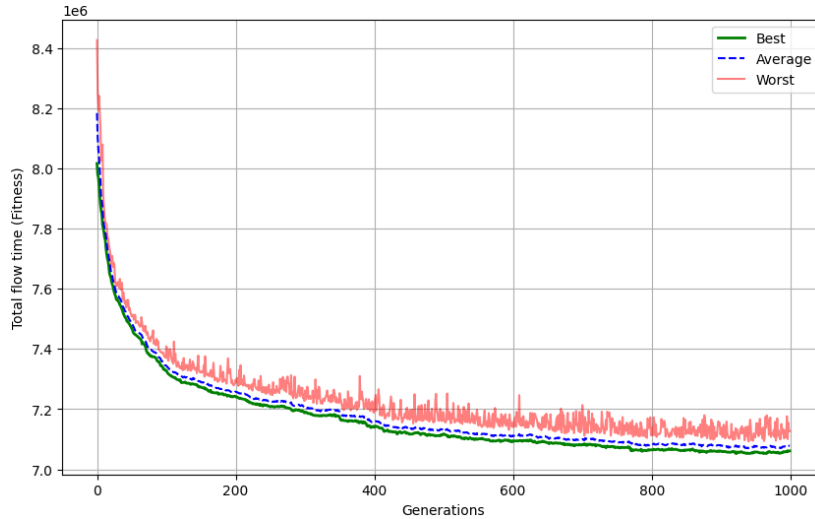


Figure 8: Evolution of the objective function across generations for the hard instance. Instance 6: 500×20

Conclusion

Our main goal was to find a way to solve the PFSP. Regarding our results, we can conclude that:

- for basic instances, the evolutionary algorithm with our final set of parameters (Table 39) is the best way to find the best solution ;
- for medium and hard instances, when the search space is bigger than the basic instance, the evolutionary algorithm has not ended its exploitation phase but the simulated annealing provides best solution in the same amount of computation time ;
- for every instance, the greedy algorithm provides the worst solution but is much faster than every other method ;
- the random search takes almost as long as the evolutionary algorithm but provides worse solution for the same number of total evaluations.

Moreover, our analysis has shown that the set of parameters is very important. Our approach was to determine a unique set of parameters for each instance. However, we can optimize the parameters depending on the instance: we can insist on exploration for hard instances and exploitation for basic instances.



References

- [1] M. R. Garey, D. S. Johnson, and Ravi Sethi. “The complexity of flowshop and jobshop scheduling”. In: *Mathematics of Operations Research* 1.2 (1976), pp. 117–129. doi: 10.1287/moor.1.2.117.
- [2] Michał Przewoźniczek et al. *Task 0: Implementation and Study of Evolutionary Algorithms Compared to Non-evolutionary Methods*. Course Assignment, Wrocław University of Science and Technology (PWr). Project description. Mar. 2026.
- [3] Agoston E Eiben and James E Smith. “What is an evolutionary algorithm?” In: *Introduction to evolutionary computing*. Springer, 2015, pp. 15–35.
- [4] Michał Przewoźniczek. *Genetic Algorithms: Basics and why does it work?* Lecture’s material. Wrocław University of Science, Technology, Department of Systems, and Computer Networks, 2026.
- [5] Contributeurs de Wikipédia. *Recuit simulé*. https://fr.wikipedia.org/w/index.php?title=Recuit_simulé&oldid=233808848. See on 2026/03/29. 2026.
- [6] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. “Optimization by simulated annealing”. In: *Science* 220.4598 (1983), pp. 671–680.
- [7] J. Lacroix. *Chaînes de Markov et Processus de Poisson*. Cours de DEA Probabilités et Applications, Université Pierre et Marie Curie (UPMC). Année universitaire 2001-2002. See on 2026/03/29. 2001. URL: <https://archive.wikiwix.com/cache/?url=http%3A%2F%2Fwww.proba.jussieu.fr%2Fcours%2Fdea%2Ftelehtml%2Fnode37.html>.
- [8] Stack Overflow contributors. *python - time.time vs. timeit.timeit*. <https://stackoverflow.com/questions/17579357/time-time-vs-timeit-timeit>. See on 2026/03/29. 2013.

Appendixes

List of Tables

1	Description of methods to be implemented. Each method is presented by its name, computational complexity (where J is the number of jobs, M the number of machines, E the total evaluations, P the population size, and G the number of generations), and a short description.	5
2	Hardware specifications of the machine used for computation.	20
3	Description of the parameters of the cases used to determine the best population/-generation ratio.	23
4	Description of the parameters of the cases used to determine the best crossover probability.	23
5	Description of the parameters of the cases used to determine the best mutation probability.	24
6	Description of the parameters of the cases used to determine the best tour size. . .	24
7	Description of the parameters of the cases used to determine the best mutation type. Comparison between swap and inversion mutation.	24
8	Description of the parameters of the cases used to determine the best crossover type.	25
9	Description of the parameters of the cases used to determine the impact of elitism.	25
10	Description of the parameters used to determine the best initial temperature for Simulated Annealing.	25
11	Description of the parameters used to determine the best cooling rate for Simulated Annealing.	26
12	Results for each case and each basic instance to get the best ratio generation/population size for the evolutionary algorithm.	27
13	Results for each case and each medium instance to get the best ratio generation/population size for the evolutionary algorithm.	27
14	Results for each case on the hard instance to get the best ratio generation/population size for the evolutionary algorithm.	28
15	Results for each case and each basic instance to get the best crossover probability for the evolutionary algorithm.	28
16	Results for each case and each medium instance to get the best crossover probability for the evolutionary algorithm.	29
17	Results for each case on the hard instance to get the best crossover probability for the evolutionary algorithm.	29
18	Results for each case and each basic instance to get the best mutation probability for the evolutionary algorithm.	30
19	Results for each case and each medium instance to get the best mutation probability for the evolutionary algorithm.	30



20	Results for each case on the hard instance to get the best mutation probability for the evolutionary algorithm.	31
21	Results for each case and each basic instance to get the best tour size for the evolutionary algorithm.	31
22	Results for each case and each medium instance to get the best tour size for the evolutionary algorithm.	32
23	Results for each case on the hard instance to get the best tour size for the evolutionary algorithm.	32
24	Results for each case and each basic instance to get the best mutation type for the evolutionary algorithm.	33
25	Results for each case and each medium instance to get the best mutation type for the evolutionary algorithm.	33
26	Results for each case on the hard instance to get the best mutation type for the evolutionary algorithm.	34
27	Results for each case and each basic instance to get the best crossover type for the evolutionary algorithm.	34
28	Results for each case and each medium instance to get the best crossover type for the evolutionary algorithm.	35
29	Results for each case on the hard instance to get the best crossover type for the evolutionary algorithm.	35
30	Results for each case and each basic instance to study the impact of elitism for the evolutionary algorithm.	36
31	Results for each case and each medium instance to study the impact of elitism for the evolutionary algorithm.	36
32	Results for each case on the hard instance to study the impact of elitism for the evolutionary algorithm.	37
33	Results for each case and each basic instance to get the best initial temperature for the simulated annealing.	37
34	Results for each case and each medium instance to get the best initial temperature for the simulated annealing.	38
35	Results for each case on the hard instance to get the best initial temperature for the simulated annealing.	38
36	Results for each case and each basic instance to get the best cooling rate for the simulated annealing.	39
37	Results for each case and each medium instance to get the best cooling rate for the simulated annealing.	39
38	Results for each case on the hard instance to get the best cooling rate for the simulated annealing.	40
39	Final set of parameters for the evolutionary algorithm.	40
40	Final set of parameters for the simulated annealing.	40

41	Comparison of all methods to solve the PFSP on basic instances. On average, the evolutionary algorithm is better and the worst is the greedy algorithm.	41
42	Comparison of all methods to solve the PFSP on medium instances. On average, the simulated annealing is better and the worst is the greedy algorithm.	41
43	Comparison of all methods to solve the PFSP on the hard instance. On average, the simulated annealing is better and the worst is the greedy algorithm.	41

List of Figures

1	A representation of an example sequence. (Source: [2])	6
2	The general scheme of an evolutionary algorithm as a flow-chart. All the steps of the EA are represented: we initialize a population (randomly), we select the parents, it might be mutation and recombination to form the child to create the new generation of the population. We loop until we run enough generations to have a good solution. The elitism is not represented in this scheme. (Source: [3])	11
3	Roulette selection scheme. (Source: [4])	12
4	Example of an CX crossover. (Source: [2])	17
5	Class diagram of the application implemented.	21
6	Evolution of the objective function across generations for basic instances.	42
7	Evolution of the objective function across generations for medium instances.	42
8	Evolution of the objective function across generations for the hard instance. Instance 6: 500×20	43

List of Algorithms

1	Random Search for the Permutation Flowshop Scheduling Problem (PFSP)	9
2	Constructive Greedy Heuristic for the PFSP	10
3	Swap Mutation Operator for the PFSP	12
4	Inversion Mutation Operator for the PFSP	13
5	Order Crossover (OX) Operator for the PFSP	14
6	Partially Mapped Crossover (PMX) Operator for the PFSP	16
7	Cycle Crossover (CX) Operator for the PFSP	17
8	Simulated Annealing for the PFSP	19

Listings

1	Allocating core 1 to run the experiment	20
2	Example of unit test for the evaluation function of a given problem	22
3	Implementation of the mutation	30

